Naija Prime School

Sprint 5 — Assessments, Results & Report Cards

Continuous assessment · Exam scores · Result computation · Term report cards

Long-form implementation walk-through

Author: Benjamin Fadina

Branch: sprint/5-results-reports

Built on: Sprint 1 identity + Sprint 2 academic domain + Sprint 3 students & parents + Sprint 4 attendance

Stack: .NET 10, Blazor Web App (Auto), EF Core 10, SQL Server, Radzen Blazor

Editor: Visual Studio Code with the C# Dev Kit

Repository: https://github.com/benjaminsqlserver/NaijaPrimeSchool

Contents

1. Sprint 5 in context

Sprint 5 closes the academic loop. The first four sprints set up identities, the academic calendar, the families that depend on the school, and the daily attendance register that says who turned up. Sprint 5 takes the work the pupils actually do — quizzes, tests, projects, exams — translates it into subject-level percentages and grades, and compiles those grades into the per-pupil end-of-term report card that goes home in the school bag.

Functionally, the sprint introduces a three-stage pipeline. Stage one is the gradebook: a SuperAdmin or HeadTeacher (or Teacher with the right scope) creates one TermAssessment row per piece of work, specifying max score and weight, then keys the per-pupil scores. Stage two is computation: a single button on the results page sums every weighted score for a (term, class), produces a SubjectResult row per pupil per subject, looks up the GradeBand, and ranks the class. Stage three is the report card: another button rolls every subject result for a pupil into a single ReportCard row, snaps in the attendance summary from sprint 4, and leaves room for the class teacher and head teacher's comments and for the affective and psychomotor ratings the Nigerian curriculum expects.

Once this sprint ships, every other module the school needs has a complete data path through the system. Fees can hang off pupils with active enrolments and now-finalised report cards. Parent portals can show the same report cards. Promotion can read the Average Percentage column. None of those features needed sprint 5 to start, but each of them now has a load-bearing dataset to read.

This document is a long-form implementation guide written in the tone of the sprint 4 guide. An engineer who has read sprints 1–4 and has the codebase checked out can recreate every change in this sprint without referring to the diff. The structure mirrors the build order: design decisions first, Domain entities next, Application contracts after that, Infrastructure (DbContext, services, seeder, migration) in the middle, then the Razor UI and navigation. Smoke-test, troubleshooting, and forward-compatibility chapters round it off.

1.1 Where this sits relative to sprint 4

Sprint 4 delivered the (Class × Date) and (TimetableEntry × Date) primitives for attendance plus the AttendanceStatus lookup. Sprint 5 reuses several of those load-bearing pieces unchanged and bolts new tables next to them. In particular:

- BaseEntity — every new entity in sprint 5 inherits it and picks up Guid Id, IAuditable, and ISoftDelete with no boilerplate.
- ApplicationDbContext.SaveChanges — the override stamps CreatedOn/By and ModifiedOn/By and rewrites Delete to IsDeleted = true. Every assessment, score, result, and report card therefore inherits auditing and soft delete with no extra code.
- Global query filters — every new entity declares HasQueryFilter(x => !x.IsDeleted), so deleted rows vanish from ordinary queries automatically.
- OperationResult / OperationResult<T> — every new service returns this for predictable success/failure responses.
- ILookupService — already had fifteen methods. Sprint 5 adds five (assessment types, grade bands, affective traits, psychomotor skills, trait ratings) without rewriting the existing ones.

- Term, SchoolClass, Subject, Student — the four big sprint 1–3 entities pick up new collection navigations only. None of their scalar columns change, so existing code that ignored these navigations continues to work.
- DailyAttendanceEntries / DailyAttendanceRegisters — the report card generator joins through these to compute days present, absent, and late at the time of generation.
- Radzen Blazor + the green/gold app.css — the new pages adopt the same .nps-page-header / .nps-card / .nps-form-grid primitives, plus a small new pair of grid styles (.nps-score-grid and .nps-trait-grid) so they read as part of the same product.

1.2 Functional scope delivered

Concretely, after this sprint a SuperAdmin or HeadTeacher (or, for the assessment pages, a Teacher) signing in to the application can:

1. Create a TermAssessment for any (Term, Class, Subject) tuple, picking an AssessmentType (CA1, CA2, Mid-Term, Assignment, Project, Exam) and setting a max score and a multiplier weight.
2. Open the score sheet for an assessment, see every actively-enrolled pupil pre-listed, key in scores or mark a pupil absent, save in bulk, and publish the assessment when ready.
3. From the Results page, pick a (Term, Class) and recompute the weighted subject totals across every assessment in scope. The service produces a SubjectResult per pupil per subject, looks up the GradeBand from the percentage, and ranks the class with dense ordering on ties.
4. Finalise individual SubjectResults so further recomputes leave them alone, or reopen them for a correction.
5. From the Report Cards page, generate or refresh the per-pupil term card for a (Term, Class). The generator reads every SubjectResult, joins to the day-level attendance counts, and ranks pupils by average percentage.
6. Open a single report card and key in the class teacher's and head teacher's comments, the next-term-begins date, and the five-point ratings for each affective trait (Punctuality, Honesty, …) and each psychomotor skill (Handwriting, Music, …).
7. Publish a report card to lock it from further edits, unpublish to amend something the head teacher caught, soft-delete a card that should not have existed at all.

1.3 Non-goals deliberately deferred

Sprint 5 deliberately stops short of several adjacent ideas that all sit on top of the schema it lays down. Each was weighed and consciously deferred:

- PDF / printed report cards. The data model is complete and the ReportCardDetailDto carries everything a printable layout needs. We will add a Razor-based print stylesheet (and likely a QuestPDF-driven server endpoint) in a follow-up sprint, since the design choices there are partly visual.
- End-of-session aggregation. A pupil's annual position and average involve summing across three Term cards, which is a second computation pass. The data is available; the rule (promote at >=50%, repeat below) is school-policy-specific and deferred until we are talking to a real school.

- Subject-by-class subject lists. Today any assessment can be created for any (class, subject) pair, even if the class does not officially offer that subject. The 'official subject list per class' will land alongside curriculum/lesson-note tooling in a later sprint.
- Per-class assessment-scheme templates. Every assessment is created individually. A 'standard scheme: CA1 + CA2 + Exam = 100' template per class level would speed setup but is a feature on top of the existing schema, not a redesign.
- Per-subject teacher comments at scale. The SubjectResult row carries a TeacherComment column; the UI exposes editing on the list page. A bulk-edit form for a (class, subject) is on the wishlist but not yet built.
- Result analytics dashboards (subject pass rates, class trends across terms). Every input is in place; the visualisations themselves are a separate piece of work.
- Parent/student portal access to results. The Parent and Student roles still see placeholder navigation; the portal sprint will expose the existing IsPublished flag as the gate for what external users can see.

1.4 Scale of the sprint

By the numbers, this sprint adds:

- 11 new domain entities under src/NaijaPrimeSchool.Domain/Results/.
- 4 collection navigations on existing entities (Subject, Term, SchoolClass, Student) so EF can navigate in the other direction; no scalar columns change on existing tables.
- 4 DTO files under src/NaijaPrimeSchool.Application/Results/Dtos/.
- 3 new service contracts under src/NaijaPrimeSchool.Application/Results/.
- 3 service implementations under src/NaijaPrimeSchool.Infrastructure/Services/.
- 5 new methods on ILookupService (and the matching LookupService).
- 1 EF Core migration introducing 11 new tables and the indexes that go with them.
- 1 DatabaseInitializer extension seeding AssessmentTypes, GradeBands, AffectiveTraits, PsychomotorSkills, and TraitRatings.
- 4 Razor pages under src/NaijaPrimeSchool.Web/Components/Pages/Results/.
- 1 navigation menu addition (Results & Reports panel) and 1 set of CSS additions (.nps-score-grid, .nps-trait-grid).
- 1 LICENSE file at the repo root — Naija Prime School is released under the MIT Licence as part of this sprint. The README's License section is updated to point at it; chapter 15 of this guide carries the rationale and the full text.

Everything compiles with zero warnings on .NET 10 (the team-wide warning bar). The code follows the patterns already accepted in sprints 1–4, so the diff is low-friction to review.

2. Design decisions and trade-offs

Before any code was written I pinned down the shape of the pipeline. Two of the calls below shape the whole feature; the rest are the kind of medium decisions that pile up when you ship a sprint and that future maintainers will thank you for documenting.

2.1 Three layers, three tables, no enums

The single biggest decision in this sprint was to model the data in three layers — assessment, subject result, report card — rather than computing report cards directly from raw scores at render time. The reasons:

- Performance. A class of 40 pupils with 8 subjects and 4 assessments per subject is 1,280 score rows. Re-aggregating that graph every time the head teacher opens a report card would be wasteful; persisting the SubjectResult is essentially a cached projection.
- Auditability. SubjectResult rows record TotalScore, Percentage, GradeBandId, Position and a FinalisedOn timestamp. If a parent queries a card three months later, the row that was persisted is the row the school stands behind — even if a teacher edits a score afterwards.
- Composability. ReportCard then hangs off SubjectResults rather than scores. The ReportCard generator's loop is simple, and the service can refuse to refresh a card whose results no longer agree.
- Narrow services. With three tables come three services (AssessmentService, ResultService, ReportCardService). Each one is easy to read and easy to change. A combined IResultsService would have grown into a kitchen sink.

The rule from earlier sprints — that domain concepts which would normally be C# enums are stored as tables — survives this sprint without exception. AssessmentType, GradeBand, AffectiveTrait, PsychomotorSkill, and TraitRating are all proper entities that derive from BaseEntity and live in the database. Schools that want a fourth assessment type, an extra grade band, or a different rating ladder change a row in a table; no recompile, no deployment.

2.2 Weighted percentage, dense ranking, no surprises

The heart of the sprint is one method on ResultService: ComputeAsync. It walks every (term, class, subject) triple in scope, sums weighted raw scores per pupil, and stamps a percentage out of the per-subject total possible. A worked example, with three assessments on Mathematics:

```
    CA1   max=20  weight=1   ->  contributes  0..20
    CA2   max=20  weight=1   ->  contributes  0..20
    Exam  max=60  weight=2   ->  contributes  0..120
    ----------------------------------------
    Total possible = 20*1 + 20*1 + 60*2 = 160
    Pupil scored CA1=18, CA2=15, Exam=40
    Weighted    = 18 + 15 + 80 = 113
    Percentage  = 113 / 160 = 70.625%  -> rounded to 70.63
```

The denominator is the per-subject 'total possible weighted', so weights act as multipliers exactly the way teachers expect. Setting every weight to 1 reduces to a simple sum-out-of-100.

Position is computed with dense ranking on the percentage column (ties share a place, the next pupil gets the next number, not a skipped one). Pupils with no enrolment in the class are filtered out before ranking begins. Students with all-zero scores still receive a percentage of 0 and a position; we do not silently drop them. The class size in the StudentsInClass column is the denominator the position is taken out of, so '5 of 38' tells the reader exactly what they think it does.

2.3 Soft-delete plus operation guards, again

Every entity in this sprint implements ISoftDelete via BaseEntity. The pattern matches sprint 4 exactly: SaveChanges intercepts EntityState.Deleted, flips IsDeleted, and stamps DeletedOn/By. Global query filters then hide the row from every subsequent query that does not call IgnoreQueryFilters.

What is genuinely new in sprint 5 is a layered set of delete and edit guards, all enforced in the services:

- TermAssessment cannot be edited or deleted while it is published. Unpublish first.
- TermAssessment cannot be deleted while it has scores attached. The service refuses with a friendly OperationResult.Failure. Clear the scores first or accept that the assessment exists in the audit trail.
- AssessmentScore is editable only while the parent assessment is in draft. Publishing locks the gradebook page to read-only.
- SubjectResult cannot be deleted while it is finalised. Reopen it first; this leaves an obvious audit trail.
- ReportCard cannot be edited or deleted while it is published. Comments, ratings, and the next-term-begins date are all rejected by the service.

2.4 Unique indexes do the heavy lifting

Three composite unique indexes are the integrity backbone of the sprint:

- (TermAssessmentId, StudentId) on AssessmentScore — a pupil's score for a given assessment is exactly one row.
- (StudentId, TermId, SubjectId) on SubjectResult — at most one computed result per (pupil, term, subject).
- (StudentId, TermId) on ReportCard — at most one card per pupil per term.

Plus (ReportCardId, AffectiveTraitId) and (ReportCardId, PsychomotorSkillId) on the rating tables, so each rating category is only ever stamped once per card. The services pre-check these unique constraints and surface friendly messages, but the database remains the authority.

2.5 Decimal precision, not floats

Every score, total, and percentage column uses HasPrecision so EF Core emits decimal columns, not the SqlServer default of decimal(18,2). Specifically:

- AssessmentScore.Score is decimal(7,2) — five digits before the point is plenty for any reasonable assessment, two after preserves half-mark resolution.
- TermAssessment.Weight is decimal(5,2) — schools rarely want a weight greater than 100, and two decimals lets you write 1.5 or 2.25.
- SubjectResult.TotalScore is decimal(7,2); SubjectResult.Percentage is decimal(5,2). Percentages can be 100.00 exactly.

- GradeBand.LowerBound and UpperBound are decimal(5,2) so a band can be specified as 70.00 to 79.99 with no round-off ambiguity.
- ReportCard.TotalScore is decimal(7,2), AveragePercentage is decimal(5,2).

2.6 Foreign-key delete behaviour

The mix is deliberate:

- AssessmentScore.TermAssessmentId -> Cascade. Wiping a draft assessment that should never have been created is allowed (in the data sense; the service still requires the assessment to have no scores in the soft-delete pre-check).
- AffectiveRating.ReportCardId / PsychomotorRating.ReportCardId -> Cascade. Removing a draft report card cleans up its trait rows. The service still refuses if the card is published.
- Lookup foreign keys (AssessmentTypeId, GradeBandId, AffectiveTraitId, PsychomotorSkillId, TraitRatingId) -> Restrict where required, SetNull where optional. You cannot accidentally lose a grade band whose name appears on existing finalised results.
- Cross-aggregate keys (StudentId, TermId, SubjectId, SchoolClassId) -> Restrict. The schema refuses to lose a subject while results still reference it. Soft-delete from the service layer is the supported path.

2.7 Inline forms again, no dialogs

The sprint 2 / 3 / 4 pattern is preserved. Every CRUD page reveals an inline RadzenCard form below its data grid rather than opening a dialog. Three reasons:

- Validation messages stay close to fields and are easy to read.
- The score sheet and trait sheet are HTML tables (.nps-score-grid and .nps-trait-grid) — flat tables hosted in a card, not Radzen-data-grid editing — because every row needs three or four widgets next to each other and Radzen's grid editor chokes on that pattern.
- Server-side rendering of a Razor page is straightforward; dialog components require extra ceremony and life-time management we do not need.

2.8 Tabs for the report card detail

ReportCardDetail.razor uses a Radzen tab strip with four tabs: Subjects (the read-only table of computed results), Affective traits (the trait × rating grid), Psychomotor skills (same shape), and Comments (free-text plus the next-term date). Splitting these prevents the page from becoming an unreadable wall of fields and keeps the user's mental task — 'I am picking ratings now' — aligned with what is on the screen.

3. The Domain layer in full

Every sprint-5 entity lives in a single new folder, src/NaijaPrimeSchool.Domain/Results/. There are no abstract base classes other than BaseEntity, no domain methods, no validation logic. Validation lives in the Application DTOs (DataAnnotations) and in the Infrastructure services (cross-aggregate checks). The Domain layer remains a typed vocabulary.

3.1 Folder layout and namespacing

```
src/NaijaPrimeSchool.Domain/
├── Results/                        <- (new in sprint 5)
│   ├── AssessmentType.cs           <- lookup
│   ├── GradeBand.cs                <- lookup
│   ├── AffectiveTrait.cs           <- lookup
│   ├── PsychomotorSkill.cs         <- lookup
│   ├── TraitRating.cs              <- lookup
│   ├── TermAssessment.cs           <- gradebook entry
│   ├── AssessmentScore.cs          <- per-pupil score
│   ├── SubjectResult.cs            <- per (pupil, term, subject) total
│   ├── ReportCard.cs               <- per (pupil, term) summary
│   ├── AffectiveRating.cs          <- card x trait rating
│   └── PsychomotorRating.cs        <- card x skill rating
├── Attendance/                     <- from sprint 4
├── Family/                         <- from sprint 3
├── Academics/                      <- from sprint 2
├── Common/                         <- from sprint 1
└── Identity/                       <- from sprint 1
```

3.2 The five lookup entities

3.2.1 AssessmentType.cs

The lookup of what kind of assessment a row represents. Each row carries a default max score (so the form can pre-fill it for new assessments) and an IsExam flag (so the UI can stamp 'Exam' badges on summative assessments).

Listing — src/NaijaPrimeSchool.Domain/Results/AssessmentType.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class AssessmentType : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DefaultMaxScore { get; set; }
    public bool IsExam { get; set; }
    public int DisplayOrder { get; set; }

    public ICollection<TermAssessment> TermAssessments { get; set; } = [];
}
```

3.2.2 GradeBand.cs

GradeBand is the lookup of grade letters with bounds. The computation finds the band whose [LowerBound, UpperBound] range contains the percentage. Description is the human-friendly label (Excellent, Very Good, …); Remark is the boilerplate one-line comment that appears on the report card.

Listing — src/NaijaPrimeSchool.Domain/Results/GradeBand.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class GradeBand : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public decimal LowerBound { get; set; }
    public decimal UpperBound { get; set; }
    public string Description { get; set; } = string.Empty;
    public string? Remark { get; set; }
    public int DisplayOrder { get; set; }

    public ICollection<SubjectResult> SubjectResults { get; set; } = [];
}
```

3.2.3 AffectiveTrait.cs and PsychomotorSkill.cs

Two parallel lookup tables for the soft-skills section of the Nigerian primary report card. AffectiveTraits cover behaviour and character (Punctuality, Honesty, Cooperation); PsychomotorSkills cover physical and creative work (Handwriting, Music, Sports). The schema is identical, just the owning collection differs.

Listing — src/NaijaPrimeSchool.Domain/Results/AffectiveTrait.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class AffectiveTrait : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<AffectiveRating> Ratings { get; set; } = [];
}
```

Listing — src/NaijaPrimeSchool.Domain/Results/PsychomotorSkill.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class PsychomotorSkill : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }
```

```
    public ICollection<PsychomotorRating> Ratings { get; set; } = [];
}
```

3.2.4 TraitRating.cs

The shared 1–5 rating ladder used for both affective and psychomotor entries. Both navigation collections live here so the schema stays consistent.

Listing — src/NaijaPrimeSchool.Domain/Results/TraitRating.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class TraitRating : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public int Value { get; set; }
    public int DisplayOrder { get; set; }

    public ICollection<AffectiveRating> AffectiveRatings { get; set; } = [];
    public ICollection<PsychomotorRating> PsychomotorRatings { get; set; } = [];
}
```

3.3 The six core entities

3.3.1 TermAssessment.cs

TermAssessment is the gradebook entry: a single quiz, project, or exam tied to a (Term, Class, Subject) tuple. MaxScore and Weight together govern how the assessment contributes to the subject total during computation. IsPublished and PublishedOn drive the gradebook lock — once published, the score sheet is read-only.

Listing — src/NaijaPrimeSchool.Domain/Results/TermAssessment.cs

```
using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class TermAssessment : BaseEntity
{
    public Guid TermId { get; set; }
    public Term? Term { get; set; }

    public Guid SchoolClassId { get; set; }
    public SchoolClass? SchoolClass { get; set; }

    public Guid SubjectId { get; set; }
    public Subject? Subject { get; set; }

    public Guid AssessmentTypeId { get; set; }
    public AssessmentType? AssessmentType { get; set; }
```

```
    public string Title { get; set; } = string.Empty;
    public int MaxScore { get; set; }
    public decimal Weight { get; set; } = 1m;
    public DateOnly? AssessmentDate { get; set; }

    public bool IsPublished { get; set; }
    public DateTimeOffset? PublishedOn { get; set; }

    public string? Notes { get; set; }

    public ICollection<AssessmentScore> Scores { get; set; } = [];
}
```

3.3.2 AssessmentScore.cs

AssessmentScore is the per-pupil score on a given assessment. Score is nullable so an absent pupil can be recorded with IsAbsent = true and Score = null without polluting the computation. Remarks lets a teacher annotate a particular score (e.g. 'sat the make-up exam', 'partial credit only').

Listing — src/NaijaPrimeSchool.Domain/Results/AssessmentScore.cs

```
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;

namespace NaijaPrimeSchool.Domain.Results;

public class AssessmentScore : BaseEntity
{
    public Guid TermAssessmentId { get; set; }
    public TermAssessment? TermAssessment { get; set; }

    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public decimal? Score { get; set; }
    public bool IsAbsent { get; set; }
    public string? Remarks { get; set; }
}
```

3.3.3 SubjectResult.cs

SubjectResult is the persisted output of ResultService.ComputeAsync. TotalScore is the weighted total; Percentage is TotalScore / TotalPossibleWeighted * 100; GradeBandId is the lookup match for that percentage; Position is the dense rank in the (subject, class, term) cohort. IsFinalised + FinalisedOn lock the row from being recomputed without an explicit reopen.

Listing — src/NaijaPrimeSchool.Domain/Results/SubjectResult.cs

```
using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;
```

```
namespace NaijaPrimeSchool.Domain.Results;

public class SubjectResult : BaseEntity
{
    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public Guid TermId { get; set; }
    public Term? Term { get; set; }

    public Guid SubjectId { get; set; }
    public Subject? Subject { get; set; }

    public Guid SchoolClassId { get; set; }
    public SchoolClass? SchoolClass { get; set; }

    public decimal TotalScore { get; set; }
    public decimal Percentage { get; set; }

    public Guid? GradeBandId { get; set; }
    public GradeBand? GradeBand { get; set; }

    public int? Position { get; set; }
    public int? StudentsInClass { get; set; }

    public string? TeacherComment { get; set; }

    public bool IsFinalised { get; set; }
    public DateTimeOffset? FinalisedOn { get; set; }
}
```

3.3.4 ReportCard.cs

ReportCard is the per-(pupil, term) roll-up. Carries the average across the pupil's subjects, the position in class, the attendance counts (read from sprint 4 at generation time), the two free-text comments, the next-term date, and the publishing flag. Two collection navigations dangle off it: the affective and psychomotor ratings.

Listing — src/NaijaPrimeSchool.Domain/Results/ReportCard.cs

```
using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;

namespace NaijaPrimeSchool.Domain.Results;

public class ReportCard : BaseEntity
{
    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public Guid TermId { get; set; }
    public Term? Term { get; set; }
```

```
    public Guid SchoolClassId { get; set; }
    public SchoolClass? SchoolClass { get; set; }

    public int SubjectsTaken { get; set; }
    public decimal TotalScore { get; set; }
    public decimal AveragePercentage { get; set; }

    public int? Position { get; set; }
    public int? StudentsInClass { get; set; }

    public int DaysPresent { get; set; }
    public int DaysAbsent { get; set; }
    public int DaysLate { get; set; }
    public int TotalSchoolDays { get; set; }

    public string? ClassTeacherComment { get; set; }
    public string? HeadTeacherComment { get; set; }

    public DateOnly? NextTermBegins { get; set; }

    public bool IsPublished { get; set; }
    public DateTimeOffset? PublishedOn { get; set; }

    public ICollection<AffectiveRating> AffectiveRatings { get; set; } = [];
    public ICollection<PsychomotorRating> PsychomotorRatings { get; set; } = [];
}
```

3.3.5 AffectiveRating.cs and PsychomotorRating.cs

Two parallel join entities. Each one ties a specific report card to a specific trait/skill, with the chosen TraitRating. The schema enforces uniqueness on (ReportCardId, TraitId) and (ReportCardId, SkillId) respectively, so each row in the report-card UI grid maps to exactly one row.

Listing — src/NaijaPrimeSchool.Domain/Results/AffectiveRating.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Results;

public class AffectiveRating : BaseEntity
{
    public Guid ReportCardId { get; set; }
    public ReportCard? ReportCard { get; set; }

    public Guid AffectiveTraitId { get; set; }
    public AffectiveTrait? AffectiveTrait { get; set; }

    public Guid TraitRatingId { get; set; }
    public TraitRating? TraitRating { get; set; }
}
```

Listing — src/NaijaPrimeSchool.Domain/Results/PsychomotorRating.cs

```
using NaijaPrimeSchool.Domain.Common;
```

```
namespace NaijaPrimeSchool.Domain.Results;

public class PsychomotorRating : BaseEntity
{
    public Guid ReportCardId { get; set; }
    public ReportCard? ReportCard { get; set; }

    public Guid PsychomotorSkillId { get; set; }
    public PsychomotorSkill? PsychomotorSkill { get; set; }

    public Guid TraitRatingId { get; set; }
    public TraitRating? TraitRating { get; set; }
}
```

3.4 Back-references on existing entities

Four sprint-1-to-4 entities pick up new collection navigations so EF can navigate the new graph in both directions. None of their scalar columns change, so existing code that ignored these properties continues to compile unchanged.

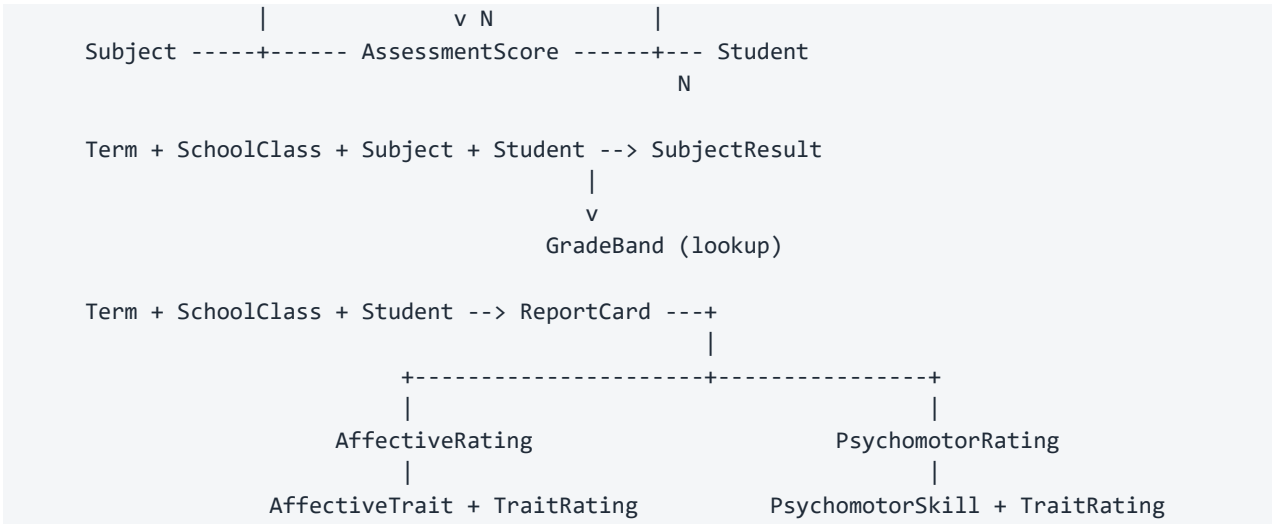
- Subject — TermAssessments and SubjectResults.
- Term — TermAssessments, SubjectResults, and ReportCards.
- SchoolClass — TermAssessments, SubjectResults, and ReportCards.
- Student — AssessmentScores, SubjectResults, and ReportCards.

Listing — src/NaijaPrimeSchool.Domain/Academics/Subject.cs

```
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Results;

namespace NaijaPrimeSchool.Domain.Academics;

public class Subject : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public string? Description { get; set; }

    public ICollection<TimetableEntry> TimetableEntries { get; set; } = [];
    public ICollection<TermAssessment> TermAssessments { get; set; } = [];
    public ICollection<SubjectResult> SubjectResults { get; set; } = [];
}
```

3.5 Relationships at a glance

```
                        AssessmentType (lookup)
                                 |
                                 v 1..N
    Term --------+------ TermAssessment ------+
                 |              | 1           |
    SchoolClass -+              |             |
```


| v N |
Subject -----+------ AssessmentScore ------+--- Student
N
Term + SchoolClass + Subject + Student --> SubjectResult
|
v
GradeBand (lookup)
Term + SchoolClass + Student --> ReportCard ---+
|
+----------+----------+
| |
AffectiveRating PsychomotorRating
| |
AffectiveTrait + TraitRating PsychomotorSkill + TraitRating

4. Application layer — DTOs and contracts

Application stays thin: just DTOs and service interfaces. There is no domain logic and no EF Core. Sprint 5 adds two new subfolders: Results/Dtos and Results.

4.1 DTO design rules

- Every list view returns a flat read-DTO with denormalised display fields. TermAssessmentDto carries the term/class/subject names alongside the assessment-type code so a row renders without lazy-loading any navigations.
- Computed counts (ScoredCount, ExpectedCount) are computed in the projection so the gradebook list shows progress at a glance.
- Aggregate operations (compute, generate) carry their own request/response shapes so the service signatures don't leak tuples or anonymous types.
- GradeBand is denormalised onto SubjectResultDto via GradeBandName and GradeBandRemark — the consuming Razor page is exclusively read-only for these fields, so denormalising saves one join per row.

4.2 TermAssessmentDtos.cs

Listing — src/NaijaPrimeSchool.Application/Results/Dtos/TermAssessmentDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Results.Dtos;

public class TermAssessmentDto
{
    public Guid Id { get; set; }

    public Guid TermId { get; set; }
    public string TermName { get; set; } = string.Empty;

    public Guid SessionId { get; set; }
    public string SessionName { get; set; } = string.Empty;

    public Guid SchoolClassId { get; set; }
    public string SchoolClassName { get; set; } = string.Empty;

    public Guid SubjectId { get; set; }
    public string SubjectName { get; set; } = string.Empty;
    public string SubjectCode { get; set; } = string.Empty;

    public Guid AssessmentTypeId { get; set; }
    public string AssessmentTypeName { get; set; } = string.Empty;
    public string AssessmentTypeCode { get; set; } = string.Empty;
    public bool IsExam { get; set; }

    public string Title { get; set; } = string.Empty;
    public int MaxScore { get; set; }
    public decimal Weight { get; set; }
    public DateOnly? AssessmentDate { get; set; }
```

```
    public bool IsPublished { get; set; }
    public DateTimeOffset? PublishedOn { get; set; }

    public string? Notes { get; set; }

    public int ScoredCount { get; set; }
    public int ExpectedCount { get; set; }
}

public class CreateTermAssessmentRequest
{
    [Required] public Guid TermId { get; set; }
    [Required] public Guid SchoolClassId { get; set; }
    [Required] public Guid SubjectId { get; set; }
    [Required] public Guid AssessmentTypeId { get; set; }

    [Required, StringLength(120)]
    public string Title { get; set; } = string.Empty;

    [Range(1, 1000)]
    public int MaxScore { get; set; } = 100;

    [Range(0.0, 100.0)]
    public decimal Weight { get; set; } = 1m;

    public DateOnly? AssessmentDate { get; set; }

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class UpdateTermAssessmentRequest
{
    public Guid Id { get; set; }
    [Required] public Guid AssessmentTypeId { get; set; }

    [Required, StringLength(120)]
    public string Title { get; set; } = string.Empty;

    [Range(1, 1000)]
    public int MaxScore { get; set; }

    [Range(0.0, 100.0)]
    public decimal Weight { get; set; }

    public DateOnly? AssessmentDate { get; set; }

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class TermAssessmentFilter
{
    public Guid? TermId { get; set; }
```

```
    public Guid? SchoolClassId { get; set; }
    public Guid? SubjectId { get; set; }
    public Guid? AssessmentTypeId { get; set; }
    public bool? IsPublished { get; set; }
}
```

4.3 AssessmentScoreDtos.cs

Listing — src/NaijaPrimeSchool.Application/Results/Dtos/AssessmentScoreDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Results.Dtos;

public class AssessmentScoreDto
{
    public Guid Id { get; set; }
    public Guid TermAssessmentId { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;

    public decimal? Score { get; set; }
    public bool IsAbsent { get; set; }
    public string? Remarks { get; set; }
}

public class AssessmentScoreSheetDto
{
    public TermAssessmentDto Assessment { get; set; } = new();
    public List<AssessmentScoreDto> Scores { get; set; } = [];
}

public class UpsertAssessmentScoreRequest
{
    public Guid? Id { get; set; }
    [Required] public Guid TermAssessmentId { get; set; }
    [Required] public Guid StudentId { get; set; }
    public decimal? Score { get; set; }
    public bool IsAbsent { get; set; }

    [StringLength(300)]
    public string? Remarks { get; set; }
}

public class BulkSetScoresRequest
{
    [Required] public Guid TermAssessmentId { get; set; }
    public List<UpsertAssessmentScoreRequest> Scores { get; set; } = [];
}
```

AssessmentScoreSheetDto is the shape the score-entry page consumes. It bundles the assessment metadata with the per-pupil rows so a single round-trip populates the whole UI.

4.4 SubjectResultDtos.cs

Listing — src/NaijaPrimeSchool.Application/Results/Dtos/SubjectResultDtos.cs

```
namespace NaijaPrimeSchool.Application.Results.Dtos;

public class SubjectResultDto
{
    public Guid Id { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;

    public Guid TermId { get; set; }
    public string TermName { get; set; } = string.Empty;

    public Guid SubjectId { get; set; }
    public string SubjectName { get; set; } = string.Empty;
    public string SubjectCode { get; set; } = string.Empty;

    public Guid SchoolClassId { get; set; }
    public string SchoolClassName { get; set; } = string.Empty;

    public decimal TotalScore { get; set; }
    public decimal Percentage { get; set; }

    public Guid? GradeBandId { get; set; }
    public string? GradeBandName { get; set; }
    public string? GradeBandRemark { get; set; }

    public int? Position { get; set; }
    public int? StudentsInClass { get; set; }

    public string? TeacherComment { get; set; }
    public bool IsFinalised { get; set; }
    public DateTimeOffset? FinalisedOn { get; set; }
}

public class SubjectResultFilter
{
    public Guid? StudentId { get; set; }
    public Guid? TermId { get; set; }
    public Guid? SubjectId { get; set; }
    public Guid? SchoolClassId { get; set; }
    public bool? IsFinalised { get; set; }
}

public class ComputeResultsRequest
{
    public Guid TermId { get; set; }
    public Guid SchoolClassId { get; set; }
```

```
    public Guid? SubjectId { get; set; }
    public bool Finalise { get; set; }
}

public class ComputeResultsResponse
{
    public int RowsComputed { get; set; }
    public int RowsFinalised { get; set; }
    public List<string> Warnings { get; set; } = [];
}

public class UpdateSubjectResultRequest
{
    public Guid Id { get; set; }

    public string? TeacherComment { get; set; }
}
```

Note ComputeResultsRequest with its Finalise flag — the same API supports 'recompute everything' (Finalise=false, idempotent and safe to run after every score change) and 'compute and lock' (Finalise=true, used at end of term). Already-finalised rows are left alone unless the user explicitly reopens them first; ComputeResultsResponse.Warnings carries any rows that were skipped for that reason.

4.5 ReportCardDtos.cs

Listing — src/NaijaPrimeSchool.Application/Results/Dtos/ReportCardDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Results.Dtos;

public class ReportCardDto
{
    public Guid Id { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;

    public Guid TermId { get; set; }
    public string TermName { get; set; } = string.Empty;

    public Guid SessionId { get; set; }
    public string SessionName { get; set; } = string.Empty;

    public Guid SchoolClassId { get; set; }
    public string SchoolClassName { get; set; } = string.Empty;

    public int SubjectsTaken { get; set; }
    public decimal TotalScore { get; set; }
    public decimal AveragePercentage { get; set; }

    public int? Position { get; set; }
```

```
    public int? StudentsInClass { get; set; }

    public int DaysPresent { get; set; }
    public int DaysAbsent { get; set; }
    public int DaysLate { get; set; }
    public int TotalSchoolDays { get; set; }

    public string? ClassTeacherComment { get; set; }
    public string? HeadTeacherComment { get; set; }

    public DateOnly? NextTermBegins { get; set; }

    public bool IsPublished { get; set; }
    public DateTimeOffset? PublishedOn { get; set; }
}

public class AffectiveRatingDto
{
    public Guid Id { get; set; }
    public Guid AffectiveTraitId { get; set; }
    public string AffectiveTraitName { get; set; } = string.Empty;
    public Guid TraitRatingId { get; set; }
    public string TraitRatingName { get; set; } = string.Empty;
    public int TraitRatingValue { get; set; }
}

public class PsychomotorRatingDto
{
    public Guid Id { get; set; }
    public Guid PsychomotorSkillId { get; set; }
    public string PsychomotorSkillName { get; set; } = string.Empty;
    public Guid TraitRatingId { get; set; }
    public string TraitRatingName { get; set; } = string.Empty;
    public int TraitRatingValue { get; set; }
}

public class ReportCardDetailDto
{
    public ReportCardDto Card { get; set; } = new();
    public List<SubjectResultDto> Results { get; set; } = [];
    public List<AffectiveRatingDto> AffectiveRatings { get; set; } = [];
    public List<PsychomotorRatingDto> PsychomotorRatings { get; set; } = [];
}

public class GenerateReportCardsRequest
{
    [Required] public Guid TermId { get; set; }
    [Required] public Guid SchoolClassId { get; set; }
    public DateOnly? NextTermBegins { get; set; }
}

public class GenerateReportCardsResponse
{
    public int CardsGenerated { get; set; }
```

```
    public int CardsUpdated { get; set; }
    public List<string> Warnings { get; set; } = [];
}

public class UpdateReportCardCommentsRequest
{
    public Guid Id { get; set; }

    [StringLength(1000)]
    public string? ClassTeacherComment { get; set; }

    [StringLength(1000)]
    public string? HeadTeacherComment { get; set; }

    public DateOnly? NextTermBegins { get; set; }
}

public class UpsertAffectiveRatingRequest
{
    [Required] public Guid ReportCardId { get; set; }
    [Required] public Guid AffectiveTraitId { get; set; }
    [Required] public Guid TraitRatingId { get; set; }
}

public class UpsertPsychomotorRatingRequest
{
    [Required] public Guid ReportCardId { get; set; }
    [Required] public Guid PsychomotorSkillId { get; set; }
    [Required] public Guid TraitRatingId { get; set; }
}

public class ReportCardFilter
{
    public Guid? TermId { get; set; }
    public Guid? SchoolClassId { get; set; }
    public Guid? StudentId { get; set; }
    public bool? IsPublished { get; set; }
}
```

ReportCardDetailDto is the read-shape the detail page asks for: the card itself, the list of subject results, plus the affective/psychomotor rating grids. Each grid row carries the trait/skill name and the rating name and value so the UI can render without any extra lookups.

4.6 Service contracts

Three new contracts. Each one is small and reads top-to-bottom as the lifecycle of one entity.

Listing — src/NaijaPrimeSchool.Application/Results/IAssessmentService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Results.Dtos;

namespace NaijaPrimeSchool.Application.Results;
```

```
public interface IAssessmentService
{
    Task<IReadOnlyList<TermAssessmentDto>> ListAsync(TermAssessmentFilter filter,
CancellationToken ct = default);
    Task<TermAssessmentDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> CreateAsync(CreateTermAssessmentRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpdateAsync(UpdateTermAssessmentRequest request,
CancellationToken ct = default);
    Task<OperationResult> PublishAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> UnpublishAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<AssessmentScoreSheetDto?> GetScoreSheetAsync(Guid assessmentId, CancellationToken
ct = default);
    Task<OperationResult> UpsertScoreAsync(UpsertAssessmentScoreRequest request,
CancellationToken ct = default);
    Task<OperationResult> BulkSetScoresAsync(BulkSetScoresRequest request,
CancellationToken ct = default);
}
```

Listing — src/NaijaPrimeSchool.Application/Results/IResultService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Results.Dtos;

namespace NaijaPrimeSchool.Application.Results;

public interface IResultService
{
    Task<IReadOnlyList<SubjectResultDto>> ListAsync(SubjectResultFilter filter,
CancellationToken ct = default);
    Task<SubjectResultDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<ComputeResultsResponse>> ComputeAsync(ComputeResultsRequest
request, CancellationToken ct = default);
    Task<OperationResult> UpdateCommentAsync(UpdateSubjectResultRequest request,
CancellationToken ct = default);
    Task<OperationResult> FinaliseAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> ReopenAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);
}
```

Listing — src/NaijaPrimeSchool.Application/Results/IReportCardService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Results.Dtos;

namespace NaijaPrimeSchool.Application.Results;

public interface IReportCardService
{
    Task<IReadOnlyList<ReportCardDto>> ListAsync(ReportCardFilter filter, CancellationToken
```

```
ct = default);
    Task<ReportCardDetailDto?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<ReportCardDetailDto?> GetForStudentTermAsync(Guid studentId, Guid termId,
CancellationToken ct = default);

    Task<OperationResult<GenerateReportCardsResponse>>
GenerateAsync(GenerateReportCardsRequest request, CancellationToken ct = default);

    Task<OperationResult> UpdateCommentsAsync(UpdateReportCardCommentsRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpsertAffectiveRatingAsync(UpsertAffectiveRatingRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpsertPsychomotorRatingAsync(UpsertPsychomotorRatingRequest
request, CancellationToken ct = default);

    Task<OperationResult> PublishAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> UnpublishAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);
}
```

4.7 ILookupService extension

ILookupService grew by five methods, one per new lookup table.

Excerpt — ILookupService.cs (sprint 5 additions)

```
    Task<IReadOnlyList<LookupDto>> GetAssessmentTypesAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetGradeBandsAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetAffectiveTraitsAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetPsychomotorSkillsAsync(CancellationToken ct =
default);
    Task<IReadOnlyList<LookupDto>> GetTraitRatingsAsync(CancellationToken ct = default);
```

5. Infrastructure — DbContext changes

ApplicationDbContext picks up eleven new DbSets, one new ConfigureResults method, and one extra invocation in OnModelCreating. Everything else in the file is unchanged from sprint 4.

5.1 New DbSets

Excerpt — the eleven results DbSets

```
    public DbSet<AssessmentType> AssessmentTypes => Set<AssessmentType>();
    public DbSet<GradeBand> GradeBands => Set<GradeBand>();
    public DbSet<AffectiveTrait> AffectiveTraits => Set<AffectiveTrait>();
    public DbSet<PsychomotorSkill> PsychomotorSkills => Set<PsychomotorSkill>();
    public DbSet<TraitRating> TraitRatings => Set<TraitRating>();
    public DbSet<TermAssessment> TermAssessments => Set<TermAssessment>();
    public DbSet<AssessmentScore> AssessmentScores => Set<AssessmentScore>();
    public DbSet<SubjectResult> SubjectResults => Set<SubjectResult>();
    public DbSet<ReportCard> ReportCards => Set<ReportCard>();
    public DbSet<AffectiveRating> AffectiveRatings => Set<AffectiveRating>();
    public DbSet<PsychomotorRating> PsychomotorRatings => Set<PsychomotorRating>();
```

5.2 ConfigureResults

ConfigureResults is invoked from OnModelCreating after ConfigureAttendance. Like its sprint 3 and 4 siblings, it uses the ConfigureLookup<T> helper for the five lookup tables, then configures each big entity one block at a time.

Excerpt — ConfigureResults

```
    private static void ConfigureResults(ModelBuilder builder)
    {
        ConfigureLookup<AssessmentType>(builder, "AssessmentTypes", extra: b =>
        {
            b.Property(t => t.Name).HasMaxLength(50).IsRequired();
            b.Property(t => t.Code).HasMaxLength(10).IsRequired();
            b.HasIndex(t => t.Name).IsUnique();
            b.HasIndex(t => t.Code).IsUnique();
        });

        ConfigureLookup<GradeBand>(builder, "GradeBands", extra: b =>
        {
            b.Property(g => g.Name).HasMaxLength(20).IsRequired();
            b.Property(g => g.Description).HasMaxLength(80).IsRequired();
            b.Property(g => g.Remark).HasMaxLength(120);
            b.Property(g => g.LowerBound).HasPrecision(5, 2);
            b.Property(g => g.UpperBound).HasPrecision(5, 2);
            b.HasIndex(g => g.Name).IsUnique();
        });

        ConfigureLookup<AffectiveTrait>(builder, "AffectiveTraits", extra: b =>
        {
            b.Property(t => t.Name).HasMaxLength(60).IsRequired();
            b.HasIndex(t => t.Name).IsUnique();
        });
```

```
        ConfigureLookup<PsychomotorSkill>(builder, "PsychomotorSkills", extra: b =>
        {
            b.Property(s => s.Name).HasMaxLength(60).IsRequired();
            b.HasIndex(s => s.Name).IsUnique();
        });

        ConfigureLookup<TraitRating>(builder, "TraitRatings", extra: b =>
        {
            b.Property(r => r.Name).HasMaxLength(40).IsRequired();
            b.HasIndex(r => r.Name).IsUnique();
            b.HasIndex(r => r.Value).IsUnique();
        });

        builder.Entity<TermAssessment>(b =>
        {
            b.ToTable("TermAssessments");
            b.HasKey(a => a.Id);
            b.Property(a => a.Title).HasMaxLength(120).IsRequired();
            b.Property(a => a.Notes).HasMaxLength(500);
            b.Property(a => a.Weight).HasPrecision(5, 2);
            b.Property(a => a.CreatedBy).HasMaxLength(100);
            b.Property(a => a.ModifiedBy).HasMaxLength(100);
            b.Property(a => a.DeletedBy).HasMaxLength(100);

            b.HasOne(a => a.Term).WithMany(t => t.TermAssessments)
                .HasForeignKey(a => a.TermId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(a => a.SchoolClass).WithMany(c => c.TermAssessments)
                .HasForeignKey(a => a.SchoolClassId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(a => a.Subject).WithMany(s => s.TermAssessments)
                .HasForeignKey(a => a.SubjectId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(a => a.AssessmentType).WithMany(t => t.TermAssessments)
                .HasForeignKey(a => a.AssessmentTypeId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasIndex(a => new { a.TermId, a.SchoolClassId, a.SubjectId });
            b.HasIndex(a => a.IsDeleted);
            b.HasQueryFilter(a => !a.IsDeleted);
        });

        builder.Entity<AssessmentScore>(b =>
        {
            b.ToTable("AssessmentScores");
            b.HasKey(s => s.Id);
            b.Property(s => s.Score).HasPrecision(7, 2);
            b.Property(s => s.Remarks).HasMaxLength(300);
            b.Property(s => s.CreatedBy).HasMaxLength(100);
            b.Property(s => s.ModifiedBy).HasMaxLength(100);
            b.Property(s => s.DeletedBy).HasMaxLength(100);
```

```
        b.HasOne(s => s.TermAssessment).WithMany(a => a.Scores)
            .HasForeignKey(s => s.TermAssessmentId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasOne(s => s.Student).WithMany(st => st.AssessmentScores)
            .HasForeignKey(s => s.StudentId)
            .OnDelete(DeleteBehavior.Restrict);

        // One score per (assessment, student).
        b.HasIndex(s => new { s.TermAssessmentId, s.StudentId }).IsUnique();
        b.HasIndex(s => s.IsDeleted);
        b.HasQueryFilter(s => !s.IsDeleted);
    });

    builder.Entity<SubjectResult>(b =>
    {
        b.ToTable("SubjectResults");
        b.HasKey(r => r.Id);
        b.Property(r => r.TotalScore).HasPrecision(7, 2);
        b.Property(r => r.Percentage).HasPrecision(5, 2);
        b.Property(r => r.TeacherComment).HasMaxLength(500);
        b.Property(r => r.CreatedBy).HasMaxLength(100);
        b.Property(r => r.ModifiedBy).HasMaxLength(100);
        b.Property(r => r.DeletedBy).HasMaxLength(100);

        b.HasOne(r => r.Student).WithMany(s => s.SubjectResults)
            .HasForeignKey(r => r.StudentId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(r => r.Term).WithMany(t => t.SubjectResults)
            .HasForeignKey(r => r.TermId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(r => r.Subject).WithMany(s => s.SubjectResults)
            .HasForeignKey(r => r.SubjectId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(r => r.SchoolClass).WithMany(c => c.SubjectResults)
            .HasForeignKey(r => r.SchoolClassId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(r => r.GradeBand).WithMany(g => g.SubjectResults)
            .HasForeignKey(r => r.GradeBandId)
            .OnDelete(DeleteBehavior.SetNull);

        // One result per (student, term, subject).
        b.HasIndex(r => new { r.StudentId, r.TermId, r.SubjectId }).IsUnique();
        b.HasIndex(r => new { r.SchoolClassId, r.TermId, r.SubjectId });
        b.HasIndex(r => r.IsDeleted);
        b.HasQueryFilter(r => !r.IsDeleted);
    });

    builder.Entity<ReportCard>(b =>
```

```
        {
            b.ToTable("ReportCards");
            b.HasKey(c => c.Id);
            b.Property(c => c.TotalScore).HasPrecision(7, 2);
            b.Property(c => c.AveragePercentage).HasPrecision(5, 2);
            b.Property(c => c.ClassTeacherComment).HasMaxLength(1000);
            b.Property(c => c.HeadTeacherComment).HasMaxLength(1000);
            b.Property(c => c.CreatedBy).HasMaxLength(100);
            b.Property(c => c.ModifiedBy).HasMaxLength(100);
            b.Property(c => c.DeletedBy).HasMaxLength(100);

            b.HasOne(c => c.Student).WithMany(s => s.ReportCards)
                .HasForeignKey(c => c.StudentId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(c => c.Term).WithMany(t => t.ReportCards)
                .HasForeignKey(c => c.TermId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(c => c.SchoolClass).WithMany(sc => sc.ReportCards)
                .HasForeignKey(c => c.SchoolClassId)
                .OnDelete(DeleteBehavior.Restrict);

            // One report card per (student, term).
            b.HasIndex(c => new { c.StudentId, c.TermId }).IsUnique();
            b.HasIndex(c => new { c.SchoolClassId, c.TermId });
            b.HasIndex(c => c.IsDeleted);
            b.HasQueryFilter(c => !c.IsDeleted);
        });

        builder.Entity<AffectiveRating>(b =>
        {
            b.ToTable("AffectiveRatings");
            b.HasKey(r => r.Id);
            b.Property(r => r.CreatedBy).HasMaxLength(100);
            b.Property(r => r.ModifiedBy).HasMaxLength(100);
            b.Property(r => r.DeletedBy).HasMaxLength(100);

            b.HasOne(r => r.ReportCard).WithMany(c => c.AffectiveRatings)
                .HasForeignKey(r => r.ReportCardId)
                .OnDelete(DeleteBehavior.Cascade);

            b.HasOne(r => r.AffectiveTrait).WithMany(t => t.Ratings)
                .HasForeignKey(r => r.AffectiveTraitId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(r => r.TraitRating).WithMany(tr => tr.AffectiveRatings)
                .HasForeignKey(r => r.TraitRatingId)
                .OnDelete(DeleteBehavior.Restrict);

            // One rating per (report card, trait).
            b.HasIndex(r => new { r.ReportCardId, r.AffectiveTraitId }).IsUnique();
            b.HasIndex(r => r.IsDeleted);
            b.HasQueryFilter(r => !r.IsDeleted);
```

```
        });

        builder.Entity<PsychomotorRating>(b =>
        {
            b.ToTable("PsychomotorRatings");
            b.HasKey(r => r.Id);
            b.Property(r => r.CreatedBy).HasMaxLength(100);
            b.Property(r => r.ModifiedBy).HasMaxLength(100);
            b.Property(r => r.DeletedBy).HasMaxLength(100);

            b.HasOne(r => r.ReportCard).WithMany(c => c.PsychomotorRatings)
                .HasForeignKey(r => r.ReportCardId)
                .OnDelete(DeleteBehavior.Cascade);

            b.HasOne(r => r.PsychomotorSkill).WithMany(s => s.Ratings)
                .HasForeignKey(r => r.PsychomotorSkillId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(r => r.TraitRating).WithMany(tr => tr.PsychomotorRatings)
                .HasForeignKey(r => r.TraitRatingId)
                .OnDelete(DeleteBehavior.Restrict);

            // One rating per (report card, skill).
            b.HasIndex(r => new { r.ReportCardId, r.PsychomotorSkillId }).IsUnique();
            b.HasIndex(r => r.IsDeleted);
            b.HasQueryFilter(r => !r.IsDeleted);
        });
    }

    private static void ConfigureLookup<TEntity>(
```

Things worth dwelling on:

- HasPrecision on every score, total, and percentage column so the schema uses decimal(p,s) — see chapter 2.5.
- Cascade delete on AssessmentScore.TermAssessmentId, AffectiveRating.ReportCardId, and PsychomotorRating.ReportCardId so wiping a draft assessment or draft card cleans up the children. The service still pre-checks for the right draft status before letting that happen.
- Restrict on every cross-aggregate FK (StudentId, TermId, SubjectId, SchoolClassId) — the schema refuses to drop a subject while results exist for it. Soft-delete from the service is the supported path.
- Composite unique indexes on (TermAssessmentId, StudentId), (StudentId, TermId, SubjectId), (StudentId, TermId), (ReportCardId, AffectiveTraitId), (ReportCardId, PsychomotorSkillId) — these are the five rules the schema considers inviolable.

5.3 The ConfigureLookup helper, reused

Same generic helper from sprints 1–4. Each new lookup picks up table name, primary key, audit columns, and the global soft-delete query filter — the only customisation per lookup is the Name column constraint and any extra unique indexes.

6. Infrastructure — service implementations

Three new services land in src/NaijaPrimeSchool.Infrastructure/Services/. Each one implements the matching interface, leans on db.SaveChangesAsync for auditing and soft delete, and uses OperationResult to surface friendly errors.

6.1 AssessmentService.cs

The gradebook service. CRUD over TermAssessment plus the score-sheet operations. Two design notes:

- GetScoreSheetAsync pre-loads every actively-enrolled pupil for the assessment's class and matches them with any existing scores. Pupils with no score yet still appear in the response, with Id = Guid.Empty so the UI knows the row is new.
- BulkSetScoresAsync skips rows with no score and no IsAbsent and no remarks — keying nothing into a row should not create an empty AssessmentScore.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/AssessmentService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Results;
using NaijaPrimeSchool.Application.Results.Dtos;
using NaijaPrimeSchool.Domain.Results;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class AssessmentService(ApplicationDbContext db) : IAssessmentService
{
    private IQueryable<TermAssessmentDto> Project(IQueryable<TermAssessment> q) =>
        q.Select(a => new TermAssessmentDto
        {
            Id = a.Id,
            TermId = a.TermId,
            TermName = a.Term!.TermType!.Name + " — " + a.Term!.Session!.Name,
            SessionId = a.Term!.SessionId,
            SessionName = a.Term!.Session!.Name,
            SchoolClassId = a.SchoolClassId,
            SchoolClassName = a.SchoolClass!.Name,
            SubjectId = a.SubjectId,
            SubjectName = a.Subject!.Name,
            SubjectCode = a.Subject!.Code,
            AssessmentTypeId = a.AssessmentTypeId,
            AssessmentTypeName = a.AssessmentType!.Name,
            AssessmentTypeCode = a.AssessmentType!.Code,
            IsExam = a.AssessmentType!.IsExam,
            Title = a.Title,
            MaxScore = a.MaxScore,
            Weight = a.Weight,
            AssessmentDate = a.AssessmentDate,
            IsPublished = a.IsPublished,
            PublishedOn = a.PublishedOn,
            Notes = a.Notes,
            ScoredCount = a.Scores.Count,
```

```
            ExpectedCount = db.Enrolments
                .Count(e => e.SchoolClassId == a.SchoolClassId && e.WithdrawnOn == null),
        });

    public async Task<IReadOnlyList<TermAssessmentDto>> ListAsync(TermAssessmentFilter
filter, CancellationToken ct = default)
    {
        var q = db.TermAssessments.AsQueryable();
        if (filter.TermId.HasValue) q = q.Where(a => a.TermId == filter.TermId.Value);
        if (filter.SchoolClassId.HasValue) q = q.Where(a => a.SchoolClassId ==
filter.SchoolClassId.Value);
        if (filter.SubjectId.HasValue) q = q.Where(a => a.SubjectId ==
filter.SubjectId.Value);
        if (filter.AssessmentTypeId.HasValue) q = q.Where(a => a.AssessmentTypeId ==
filter.AssessmentTypeId.Value);
        if (filter.IsPublished.HasValue) q = q.Where(a => a.IsPublished ==
filter.IsPublished.Value);

        return await Project(q
                .OrderBy(a => a.Subject!.Name)
                .ThenBy(a => a.AssessmentType!.DisplayOrder)
                .ThenByDescending(a => a.AssessmentDate))
            .ToListAsync(ct);
    }

    public Task<TermAssessmentDto?> GetByIdAsync(Guid id, CancellationToken ct = default)
=>
        Project(db.TermAssessments.Where(a => a.Id == id)).FirstOrDefaultAsync(ct);

    public async Task<OperationResult<Guid>> CreateAsync(CreateTermAssessmentRequest
request, CancellationToken ct = default)
    {
        if (!await db.Terms.AnyAsync(t => t.Id == request.TermId, ct))
            return OperationResult<Guid>.Failure("Term not found.");

        if (!await db.SchoolClasses.AnyAsync(c => c.Id == request.SchoolClassId, ct))
            return OperationResult<Guid>.Failure("Class not found.");

        if (!await db.Subjects.AnyAsync(s => s.Id == request.SubjectId, ct))
            return OperationResult<Guid>.Failure("Subject not found.");

        if (!await db.AssessmentTypes.AnyAsync(t => t.Id == request.AssessmentTypeId, ct))
            return OperationResult<Guid>.Failure("Assessment type not found.");

        if (request.MaxScore <= 0)
            return OperationResult<Guid>.Failure("Maximum score must be greater than
zero.");

        if (request.Weight < 0)
            return OperationResult<Guid>.Failure("Weight cannot be negative.");

        var assessment = new TermAssessment
        {
            TermId = request.TermId,
```

```
            SchoolClassId = request.SchoolClassId,
            SubjectId = request.SubjectId,
            AssessmentTypeId = request.AssessmentTypeId,
            Title = request.Title.Trim(),
            MaxScore = request.MaxScore,
            Weight = request.Weight,
            AssessmentDate = request.AssessmentDate,
            Notes = request.Notes,
        };

        db.TermAssessments.Add(assessment);
        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(assessment.Id);
    }

    public async Task<OperationResult> UpdateAsync(UpdateTermAssessmentRequest request,
CancellationToken ct = default)
    {
        var assessment = await db.TermAssessments.FirstOrDefaultAsync(a => a.Id ==
request.Id, ct);
        if (assessment is null) return OperationResult.Failure("Assessment not found.");

        if (assessment.IsPublished)
            return OperationResult.Failure("Unpublish the assessment before editing.");

        if (!await db.AssessmentTypes.AnyAsync(t => t.Id == request.AssessmentTypeId, ct))
            return OperationResult.Failure("Assessment type not found.");

        if (request.MaxScore <= 0)
            return OperationResult.Failure("Maximum score must be greater than zero.");

        if (request.Weight < 0)
            return OperationResult.Failure("Weight cannot be negative.");

        assessment.AssessmentTypeId = request.AssessmentTypeId;
        assessment.Title = request.Title.Trim();
        assessment.MaxScore = request.MaxScore;
        assessment.Weight = request.Weight;
        assessment.AssessmentDate = request.AssessmentDate;
        assessment.Notes = request.Notes;

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> PublishAsync(Guid id, CancellationToken ct =
default)
    {
        var a = await db.TermAssessments.FirstOrDefaultAsync(x => x.Id == id, ct);
        if (a is null) return OperationResult.Failure("Assessment not found.");

        a.IsPublished = true;
        a.PublishedOn = DateTimeOffset.UtcNow;
        await db.SaveChangesAsync(ct);
```

```
        return OperationResult.Success();
    }

    public async Task<OperationResult> UnpublishAsync(Guid id, CancellationToken ct =
default)
    {
        var a = await db.TermAssessments.FirstOrDefaultAsync(x => x.Id == id, ct);
        if (a is null) return OperationResult.Failure("Assessment not found.");

        a.IsPublished = false;
        a.PublishedOn = null;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var a = await db.TermAssessments
            .Include(x => x.Scores)
            .FirstOrDefaultAsync(x => x.Id == id, ct);
        if (a is null) return OperationResult.Failure("Assessment not found.");

        if (a.IsPublished)
            return OperationResult.Failure("Unpublish the assessment before deleting.");

        if (a.Scores.Any())
            return OperationResult.Failure(
                "Cannot delete an assessment that already has scores. Clear the scores
first.");

        db.TermAssessments.Remove(a);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<AssessmentScoreSheetDto?> GetScoreSheetAsync(Guid assessmentId,
CancellationToken ct = default)
    {
        var assessment = await Project(db.TermAssessments.Where(a => a.Id == assessmentId))
            .FirstOrDefaultAsync(ct);
        if (assessment is null) return null;

        var existing = await db.AssessmentScores
            .Where(s => s.TermAssessmentId == assessmentId)
            .Select(s => new
            {
                s.Id,
                s.StudentId,
                s.Score,
                s.IsAbsent,
                s.Remarks,
            })
            .ToListAsync(ct);
```

```
        var enrolledStudents = await db.Enrolments
            .Where(e => e.SchoolClassId == assessment.SchoolClassId && e.WithdrawnOn ==
null)
            .Select(e => new
            {
                e.StudentId,
                Name = e.Student!.FirstName + " " + e.Student!.LastName,
                AdmissionNumber = e.Student!.AdmissionNumber,
            })
            .OrderBy(s => s.Name)
            .ToListAsync(ct);

        var rows = enrolledStudents
            .Select(s =>
            {
                var match = existing.FirstOrDefault(x => x.StudentId == s.StudentId);
                return new AssessmentScoreDto
                {
                    Id = match?.Id ?? Guid.Empty,
                    TermAssessmentId = assessmentId,
                    StudentId = s.StudentId,
                    StudentName = s.Name.Trim(),
                    StudentAdmissionNumber = s.AdmissionNumber,
                    Score = match?.Score,
                    IsAbsent = match?.IsAbsent ?? false,
                    Remarks = match?.Remarks,
                };
            })
            .ToList();

        return new AssessmentScoreSheetDto { Assessment = assessment, Scores = rows };
    }

    public async Task<OperationResult> UpsertScoreAsync(UpsertAssessmentScoreRequest
request, CancellationToken ct = default)
    {
        var assessment = await db.TermAssessments
            .FirstOrDefaultAsync(a => a.Id == request.TermAssessmentId, ct);
        if (assessment is null) return OperationResult.Failure("Assessment not found.");
        if (assessment.IsPublished)
            return OperationResult.Failure("Unpublish the assessment before editing
scores.");

        if (request.Score.HasValue && (request.Score.Value < 0 || request.Score.Value >
assessment.MaxScore))
            return OperationResult.Failure(
                $"Score must be between 0 and {assessment.MaxScore}.");

        var score = await db.AssessmentScores
            .FirstOrDefaultAsync(s =>
                s.TermAssessmentId == request.TermAssessmentId
                && s.StudentId == request.StudentId, ct);
```

```
        if (score is null)
        {
            score = new AssessmentScore
            {
                TermAssessmentId = request.TermAssessmentId,
                StudentId = request.StudentId,
                Score = request.IsAbsent ? null : request.Score,
                IsAbsent = request.IsAbsent,
                Remarks = request.Remarks,
            };
            db.AssessmentScores.Add(score);
        }
        else
        {
            score.Score = request.IsAbsent ? null : request.Score;
            score.IsAbsent = request.IsAbsent;
            score.Remarks = request.Remarks;
        }

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> BulkSetScoresAsync(BulkSetScoresRequest request,
CancellationToken ct = default)
    {
        var assessment = await db.TermAssessments
            .FirstOrDefaultAsync(a => a.Id == request.TermAssessmentId, ct);
        if (assessment is null) return OperationResult.Failure("Assessment not found.");
        if (assessment.IsPublished)
            return OperationResult.Failure("Unpublish the assessment before editing
scores.");

        var existing = await db.AssessmentScores
            .Where(s => s.TermAssessmentId == request.TermAssessmentId)
            .ToListAsync(ct);

        var errors = new List<string>();
        foreach (var row in request.Scores)
        {
            if (row.Score.HasValue && (row.Score.Value < 0 || row.Score.Value >
assessment.MaxScore))
            {
                errors.Add($"Student {row.StudentId}: score out of range (0..
{assessment.MaxScore}).");
                continue;
            }

            var existingRow = existing.FirstOrDefault(s => s.StudentId == row.StudentId);
            if (existingRow is null)
            {
                if (row.Score is null && !row.IsAbsent &&
string.IsNullOrWhiteSpace(row.Remarks))
                    continue; // nothing to write
```

```
                db.AssessmentScores.Add(new AssessmentScore
                {
                    TermAssessmentId = request.TermAssessmentId,
                    StudentId = row.StudentId,
                    Score = row.IsAbsent ? null : row.Score,
                    IsAbsent = row.IsAbsent,
                    Remarks = row.Remarks,
                });
            }
            else
            {
                existingRow.Score = row.IsAbsent ? null : row.Score;
                existingRow.IsAbsent = row.IsAbsent;
                existingRow.Remarks = row.Remarks;
            }
        }

        if (errors.Count > 0) return OperationResult.Failure(errors);

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }
}
```

6.2 ResultService.cs

The compute service. ComputeAsync is the longest method in the sprint and the one worth reading slowly:

8. Validate the term and class exist.
9. Pull the assessments in scope (everything in the term/class, or narrowed by subject if the request specifies one).
10. Pull the score rows for those assessments in a single query.
11. Pull the active pupil set for the class (an enrolment with the right SchoolClassId).
12. Pull all grade bands ordered by display.
13. For each subject in scope: compute the per-pupil weighted total, divide by the per-subject total possible weighted, round to two decimals, dense-rank by percentage.
14. Look up the matching grade band per pupil and write a fresh SubjectResult or update the existing one. If the existing row is finalised and the request is not asking to finalise, log a warning and skip; never silently overwrite a finalised row.
15. SaveChangesAsync once at the end. Auditing and soft-delete stamping run automatically.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/ResultService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Results;
using NaijaPrimeSchool.Application.Results.Dtos;
using NaijaPrimeSchool.Domain.Results;
using NaijaPrimeSchool.Infrastructure.Persistence;
```

```
namespace NaijaPrimeSchool.Infrastructure.Services;

public class ResultService(ApplicationDbContext db) : IResultService
{
    private static IQueryable<SubjectResultDto> Project(IQueryable<SubjectResult> q) =>
        q.Select(r => new SubjectResultDto
        {
            Id = r.Id,
            StudentId = r.StudentId,
            StudentName = (r.Student!.FirstName + " " + r.Student!.LastName).Trim(),
            StudentAdmissionNumber = r.Student!.AdmissionNumber,
            TermId = r.TermId,
            TermName = r.Term!.TermType!.Name + " — " + r.Term!.Session!.Name,
            SubjectId = r.SubjectId,
            SubjectName = r.Subject!.Name,
            SubjectCode = r.Subject!.Code,
            SchoolClassId = r.SchoolClassId,
            SchoolClassName = r.SchoolClass!.Name,
            TotalScore = r.TotalScore,
            Percentage = r.Percentage,
            GradeBandId = r.GradeBandId,
            GradeBandName = r.GradeBand == null ? null : r.GradeBand.Name,
            GradeBandRemark = r.GradeBand == null ? null : r.GradeBand.Remark,
            Position = r.Position,
            StudentsInClass = r.StudentsInClass,
            TeacherComment = r.TeacherComment,
            IsFinalised = r.IsFinalised,
            FinalisedOn = r.FinalisedOn,
        });

    public async Task<IReadOnlyList<SubjectResultDto>> ListAsync(SubjectResultFilter
filter, CancellationToken ct = default)
    {
        var q = db.SubjectResults.AsQueryable();
        if (filter.StudentId.HasValue) q = q.Where(r => r.StudentId ==
filter.StudentId.Value);
        if (filter.TermId.HasValue) q = q.Where(r => r.TermId == filter.TermId.Value);
        if (filter.SubjectId.HasValue) q = q.Where(r => r.SubjectId ==
filter.SubjectId.Value);
        if (filter.SchoolClassId.HasValue) q = q.Where(r => r.SchoolClassId ==
filter.SchoolClassId.Value);
        if (filter.IsFinalised.HasValue) q = q.Where(r => r.IsFinalised ==
filter.IsFinalised.Value);

        return await Project(q
                .OrderBy(r => r.Subject!.Name)
                .ThenBy(r => r.Position ?? int.MaxValue)
                .ThenByDescending(r => r.Percentage))
            .ToListAsync(ct);
    }

    public Task<SubjectResultDto?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        Project(db.SubjectResults.Where(r => r.Id == id)).FirstOrDefaultAsync(ct);
```

```
    public async Task<OperationResult<ComputeResultsResponse>> ComputeAsync(
        ComputeResultsRequest request, CancellationToken ct = default)
    {
        if (!await db.Terms.AnyAsync(t => t.Id == request.TermId, ct))
            return OperationResult<ComputeResultsResponse>.Failure("Term not found.");

        if (!await db.SchoolClasses.AnyAsync(c => c.Id == request.SchoolClassId, ct))
            return OperationResult<ComputeResultsResponse>.Failure("Class not found.");

        var assessmentsQuery = db.TermAssessments
            .Where(a => a.TermId == request.TermId
                && a.SchoolClassId == request.SchoolClassId);
        if (request.SubjectId.HasValue)
            assessmentsQuery = assessmentsQuery.Where(a => a.SubjectId ==
request.SubjectId.Value);

        var assessments = await assessmentsQuery
            .Select(a => new
            {
                a.Id,
                a.SubjectId,
                a.MaxScore,
                a.Weight,
            })
            .ToListAsync(ct);

        if (assessments.Count == 0)
            return OperationResult<ComputeResultsResponse>.Failure(
                "No assessments exist for this term/class. Create assessments first.");

        var scoreRows = await db.AssessmentScores
            .Where(s => assessments.Select(a => a.Id).Contains(s.TermAssessmentId))
            .Select(s => new
            {
                s.TermAssessmentId,
                s.StudentId,
                s.Score,
                s.IsAbsent,
            })
            .ToListAsync(ct);

        // Active enrolments in the class for this term — these are the candidates.
        var enrolledStudents = await db.Enrolments
            .Where(e => e.SchoolClassId == request.SchoolClassId)
            .Select(e => e.StudentId)
            .Distinct()
            .ToListAsync(ct);

        var bands = await db.GradeBands
            .OrderBy(g => g.DisplayOrder)
            .ToListAsync(ct);

        var subjectIds = assessments.Select(a => a.SubjectId).Distinct().ToList();
```

```
        var warnings = new List<string>();
        var rowsTouched = 0;
        var rowsFinalised = 0;

        // Build per-subject working data, then per-(student, subject) aggregate.
        foreach (var subjectId in subjectIds)
        {
            var subjectAssessments = assessments.Where(a => a.SubjectId ==
subjectId).ToList();
            // Total of (MaxScore × Weight) across all assessments — the denominator for
the percentage.
            decimal subjectTotalWeighted = subjectAssessments.Sum(a => a.MaxScore *
a.Weight);
            if (subjectTotalWeighted <= 0)
            {
                warnings.Add($"Subject {subjectId}: total weighted max is zero. Skipped.");
                continue;
            }

            var perStudent = new List<(Guid StudentId, decimal Total, decimal
Percentage)>();

            foreach (var studentId in enrolledStudents)
            {
                decimal studentWeighted = 0m;
                foreach (var a in subjectAssessments)
                {
                    var score = scoreRows.FirstOrDefault(s =>
                        s.TermAssessmentId == a.Id && s.StudentId == studentId);
                    decimal raw = score?.Score ?? 0m;
                    studentWeighted += raw * a.Weight;
                }

                decimal percentage = Math.Round(studentWeighted * 100m /
subjectTotalWeighted, 2);
                perStudent.Add((studentId, Math.Round(studentWeighted, 2), percentage));
            }

            // Compute positions for this subject (1-based, dense ranking on percentage).
            var ordered = perStudent
                .OrderByDescending(x => x.Percentage)
                .ToList();
            var positions = new Dictionary<Guid, int>();
            int currentPos = 0;
            decimal? prevPct = null;
            for (int i = 0; i < ordered.Count; i++)
            {
                var row = ordered[i];
                if (prevPct is null || row.Percentage != prevPct.Value)
                {
                    currentPos = i + 1;
                    prevPct = row.Percentage;
                }
                positions[row.StudentId] = currentPos;
```

```
            }

            var existingRows = await db.SubjectResults
                .Where(r => r.TermId == request.TermId
                    && r.SchoolClassId == request.SchoolClassId
                    && r.SubjectId == subjectId)
                .ToListAsync(ct);

            foreach (var row in perStudent)
            {
                var band = bands.FirstOrDefault(g =>
                    row.Percentage >= g.LowerBound && row.Percentage <= g.UpperBound);

                var existing = existingRows.FirstOrDefault(r => r.StudentId ==
row.StudentId);
                if (existing is null)
                {
                    var fresh = new SubjectResult
                    {
                        StudentId = row.StudentId,
                        TermId = request.TermId,
                        SubjectId = subjectId,
                        SchoolClassId = request.SchoolClassId,
                        TotalScore = row.Total,
                        Percentage = row.Percentage,
                        GradeBandId = band?.Id,
                        Position = positions[row.StudentId],
                        StudentsInClass = perStudent.Count,
                        IsFinalised = request.Finalise,
                        FinalisedOn = request.Finalise ? DateTimeOffset.UtcNow : null,
                    };
                    db.SubjectResults.Add(fresh);
                    rowsTouched++;
                    if (request.Finalise) rowsFinalised++;
                }
                else
                {
                    if (existing.IsFinalised && !request.Finalise)
                    {
                        warnings.Add(
                            $"Student {row.StudentId} subject {subjectId}: already
finalised — skipped recompute.");
                        continue;
                    }

                    existing.TotalScore = row.Total;
                    existing.Percentage = row.Percentage;
                    existing.GradeBandId = band?.Id;
                    existing.Position = positions[row.StudentId];
                    existing.StudentsInClass = perStudent.Count;
                    if (request.Finalise && !existing.IsFinalised)
                    {
                        existing.IsFinalised = true;
                        existing.FinalisedOn = DateTimeOffset.UtcNow;
```

```
                        rowsFinalised++;
                    }
                    rowsTouched++;
                }
            }
        }

        await db.SaveChangesAsync(ct);

        return OperationResult<ComputeResultsResponse>.Success(new ComputeResultsResponse
        {
            RowsComputed = rowsTouched,
            RowsFinalised = rowsFinalised,
            Warnings = warnings,
        });
    }

    public async Task<OperationResult> UpdateCommentAsync(UpdateSubjectResultRequest
request, CancellationToken ct = default)
    {
        var r = await db.SubjectResults.FirstOrDefaultAsync(x => x.Id == request.Id, ct);
        if (r is null) return OperationResult.Failure("Result not found.");

        r.TeacherComment = request.TeacherComment;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> FinaliseAsync(Guid id, CancellationToken ct =
default)
    {
        var r = await db.SubjectResults.FirstOrDefaultAsync(x => x.Id == id, ct);
        if (r is null) return OperationResult.Failure("Result not found.");

        r.IsFinalised = true;
        r.FinalisedOn = DateTimeOffset.UtcNow;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> ReopenAsync(Guid id, CancellationToken ct = default)
    {
        var r = await db.SubjectResults.FirstOrDefaultAsync(x => x.Id == id, ct);
        if (r is null) return OperationResult.Failure("Result not found.");

        r.IsFinalised = false;
        r.FinalisedOn = null;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
```

```
        var r = await db.SubjectResults.FirstOrDefaultAsync(x => x.Id == id, ct);
        if (r is null) return OperationResult.Failure("Result not found.");

        if (r.IsFinalised)
            return OperationResult.Failure("Cannot delete a finalised result. Reopen it
first.");

        db.SubjectResults.Remove(r);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }
}
```

6.3 ReportCardService.cs

The composer service. GenerateAsync rolls the SubjectResult set for a (term, class) into one ReportCard per pupil, joins to the DailyAttendanceEntries from sprint 4 to compute days present / absent / late, and ranks pupils by average percentage. Like ComputeAsync it never overwrites a published card; the service log entry will say 'updated' or 'skipped' and the UI surfaces the count.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/ReportCardService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Results;
using NaijaPrimeSchool.Application.Results.Dtos;
using NaijaPrimeSchool.Domain.Results;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class ReportCardService(ApplicationDbContext db) : IReportCardService
{
    private static IQueryable<ReportCardDto> ProjectCard(IQueryable<ReportCard> q) =>
        q.Select(c => new ReportCardDto
        {
            Id = c.Id,
            StudentId = c.StudentId,
            StudentName = (c.Student!.FirstName + " " + c.Student!.LastName).Trim(),
            StudentAdmissionNumber = c.Student!.AdmissionNumber,
            TermId = c.TermId,
            TermName = c.Term!.TermType!.Name + " — " + c.Term!.Session!.Name,
            SessionId = c.Term!.SessionId,
            SessionName = c.Term!.Session!.Name,
            SchoolClassId = c.SchoolClassId,
            SchoolClassName = c.SchoolClass!.Name,
            SubjectsTaken = c.SubjectsTaken,
            TotalScore = c.TotalScore,
            AveragePercentage = c.AveragePercentage,
            Position = c.Position,
            StudentsInClass = c.StudentsInClass,
            DaysPresent = c.DaysPresent,
            DaysAbsent = c.DaysAbsent,
            DaysLate = c.DaysLate,
```

```
            TotalSchoolDays = c.TotalSchoolDays,
            ClassTeacherComment = c.ClassTeacherComment,
            HeadTeacherComment = c.HeadTeacherComment,
            NextTermBegins = c.NextTermBegins,
            IsPublished = c.IsPublished,
            PublishedOn = c.PublishedOn,
        });

    public async Task<IReadOnlyList<ReportCardDto>> ListAsync(ReportCardFilter filter,
CancellationToken ct = default)
    {
        var q = db.ReportCards.AsQueryable();
        if (filter.TermId.HasValue) q = q.Where(c => c.TermId == filter.TermId.Value);
        if (filter.SchoolClassId.HasValue) q = q.Where(c => c.SchoolClassId ==
filter.SchoolClassId.Value);
        if (filter.StudentId.HasValue) q = q.Where(c => c.StudentId ==
filter.StudentId.Value);
        if (filter.IsPublished.HasValue) q = q.Where(c => c.IsPublished ==
filter.IsPublished.Value);

        return await ProjectCard(q
                .OrderBy(c => c.Position ?? int.MaxValue)
                .ThenBy(c => c.Student!.FirstName))
            .ToListAsync(ct);
    }

    public async Task<ReportCardDetailDto?> GetByIdAsync(Guid id, CancellationToken ct =
default)
    {
        var card = await ProjectCard(db.ReportCards.Where(c => c.Id == id))
            .FirstOrDefaultAsync(ct);
        if (card is null) return null;
        return await BuildDetailAsync(card, ct);
    }

    public async Task<ReportCardDetailDto?> GetForStudentTermAsync(Guid studentId, Guid
termId, CancellationToken ct = default)
    {
        var card = await ProjectCard(db.ReportCards
                .Where(c => c.StudentId == studentId && c.TermId == termId))
            .FirstOrDefaultAsync(ct);
        if (card is null) return null;
        return await BuildDetailAsync(card, ct);
    }

    private async Task<ReportCardDetailDto> BuildDetailAsync(ReportCardDto card,
CancellationToken ct)
    {
        var results = await db.SubjectResults
            .Where(r => r.StudentId == card.StudentId && r.TermId == card.TermId)
            .OrderBy(r => r.Subject!.Name)
            .Select(r => new SubjectResultDto
            {
                Id = r.Id,
```

```
            StudentId = r.StudentId,
            StudentName = card.StudentName,
            StudentAdmissionNumber = card.StudentAdmissionNumber,
            TermId = r.TermId,
            TermName = card.TermName,
            SubjectId = r.SubjectId,
            SubjectName = r.Subject!.Name,
            SubjectCode = r.Subject!.Code,
            SchoolClassId = r.SchoolClassId,
            SchoolClassName = card.SchoolClassName,
            TotalScore = r.TotalScore,
            Percentage = r.Percentage,
            GradeBandId = r.GradeBandId,
            GradeBandName = r.GradeBand == null ? null : r.GradeBand.Name,
            GradeBandRemark = r.GradeBand == null ? null : r.GradeBand.Remark,
            Position = r.Position,
            StudentsInClass = r.StudentsInClass,
            TeacherComment = r.TeacherComment,
            IsFinalised = r.IsFinalised,
            FinalisedOn = r.FinalisedOn,
        })
        .ToListAsync(ct);

    var affective = await db.AffectiveRatings
        .Where(r => r.ReportCardId == card.Id)
        .OrderBy(r => r.AffectiveTrait!.DisplayOrder)
        .Select(r => new AffectiveRatingDto
        {
            Id = r.Id,
            AffectiveTraitId = r.AffectiveTraitId,
            AffectiveTraitName = r.AffectiveTrait!.Name,
            TraitRatingId = r.TraitRatingId,
            TraitRatingName = r.TraitRating!.Name,
            TraitRatingValue = r.TraitRating!.Value,
        })
        .ToListAsync(ct);

    var psycho = await db.PsychomotorRatings
        .Where(r => r.ReportCardId == card.Id)
        .OrderBy(r => r.PsychomotorSkill!.DisplayOrder)
        .Select(r => new PsychomotorRatingDto
        {
            Id = r.Id,
            PsychomotorSkillId = r.PsychomotorSkillId,
            PsychomotorSkillName = r.PsychomotorSkill!.Name,
            TraitRatingId = r.TraitRatingId,
            TraitRatingName = r.TraitRating!.Name,
            TraitRatingValue = r.TraitRating!.Value,
        })
        .ToListAsync(ct);

    return new ReportCardDetailDto
    {
        Card = card,
        Results = results,
```

```
            AffectiveRatings = affective,
            PsychomotorRatings = psycho,
        };
    }

    public async Task<OperationResult<GenerateReportCardsResponse>> GenerateAsync(
        GenerateReportCardsRequest request, CancellationToken ct = default)
    {
        var term = await db.Terms.FirstOrDefaultAsync(t => t.Id == request.TermId, ct);
        if (term is null) return OperationResult<GenerateReportCardsResponse>.Failure("Term
not found.");

        if (!await db.SchoolClasses.AnyAsync(c => c.Id == request.SchoolClassId, ct))
            return OperationResult<GenerateReportCardsResponse>.Failure("Class not
found.");

        var resultsByStudent = await db.SubjectResults
            .Where(r => r.TermId == request.TermId && r.SchoolClassId ==
request.SchoolClassId)
            .GroupBy(r => r.StudentId)
            .Select(g => new
            {
                StudentId = g.Key,
                Subjects = g.Count(),
                TotalScore = g.Sum(r => r.Percentage),
                Average = g.Average(r => r.Percentage),
            })
            .ToListAsync(ct);

        if (resultsByStudent.Count == 0)
            return OperationResult<GenerateReportCardsResponse>.Failure(
                "No subject results exist for this term/class. Compute results first.");

        // Per-student attendance summary across this term, restricted to this class.
        var attendance = await db.DailyAttendanceEntries
            .Where(e => e.Register!.TermId == request.TermId
                && e.Register!.SchoolClassId == request.SchoolClassId)
            .GroupBy(e => e.StudentId)
            .Select(g => new
            {
                StudentId = g.Key,
                Total = g.Count(),
                Present = g.Count(x => x.AttendanceStatus!.CountsAsPresent &&
x.AttendanceStatus!.Code != "L"),
                Late = g.Count(x => x.AttendanceStatus!.Code == "L"),
                Absent = g.Count(x => !x.AttendanceStatus!.CountsAsPresent),
            })
            .ToListAsync(ct);

        var totalSchoolDays = await db.DailyAttendanceRegisters
            .Where(r => r.TermId == request.TermId && r.SchoolClassId ==
request.SchoolClassId)
            .CountAsync(ct);
```

```
        // Position by average percentage.
        var ordered = resultsByStudent
            .OrderByDescending(x => x.Average)
            .ToList();
        var positions = new Dictionary<Guid, int>();
        int currentPos = 0;
        decimal? prevAvg = null;
        for (int i = 0; i < ordered.Count; i++)
        {
            if (prevAvg is null || ordered[i].Average != prevAvg.Value)
            {
                currentPos = i + 1;
                prevAvg = ordered[i].Average;
            }
            positions[ordered[i].StudentId] = currentPos;
        }

        var existing = await db.ReportCards
            .Where(c => c.TermId == request.TermId && c.SchoolClassId ==
request.SchoolClassId)
            .ToListAsync(ct);

        var generated = 0;
        var updated = 0;

        foreach (var s in resultsByStudent)
        {
            var att = attendance.FirstOrDefault(a => a.StudentId == s.StudentId);
            var card = existing.FirstOrDefault(c => c.StudentId == s.StudentId);

            if (card is null)
            {
                card = new ReportCard
                {
                    StudentId = s.StudentId,
                    TermId = request.TermId,
                    SchoolClassId = request.SchoolClassId,
                    SubjectsTaken = s.Subjects,
                    TotalScore = Math.Round(s.TotalScore, 2),
                    AveragePercentage = Math.Round(s.Average, 2),
                    Position = positions[s.StudentId],
                    StudentsInClass = resultsByStudent.Count,
                    DaysPresent = att?.Present ?? 0,
                    DaysAbsent = att?.Absent ?? 0,
                    DaysLate = att?.Late ?? 0,
                    TotalSchoolDays = totalSchoolDays,
                    NextTermBegins = request.NextTermBegins,
                };
                db.ReportCards.Add(card);
                generated++;
            }
            else
            {
                if (card.IsPublished)
```

```
                    continue; // never overwrite a published card
                card.SubjectsTaken = s.Subjects;
                card.TotalScore = Math.Round(s.TotalScore, 2);
                card.AveragePercentage = Math.Round(s.Average, 2);
                card.Position = positions[s.StudentId];
                card.StudentsInClass = resultsByStudent.Count;
                card.DaysPresent = att?.Present ?? 0;
                card.DaysAbsent = att?.Absent ?? 0;
                card.DaysLate = att?.Late ?? 0;
                card.TotalSchoolDays = totalSchoolDays;
                if (request.NextTermBegins.HasValue) card.NextTermBegins =
request.NextTermBegins;
                updated++;
            }
        }

        await db.SaveChangesAsync(ct);

        return OperationResult<GenerateReportCardsResponse>.Success(new
GenerateReportCardsResponse
        {
            CardsGenerated = generated,
            CardsUpdated = updated,
            Warnings = [],
        });
    }

    public async Task<OperationResult> UpdateCommentsAsync(UpdateReportCardCommentsRequest
request, CancellationToken ct = default)
    {
        var card = await db.ReportCards.FirstOrDefaultAsync(c => c.Id == request.Id, ct);
        if (card is null) return OperationResult.Failure("Report card not found.");
        if (card.IsPublished) return OperationResult.Failure("Unpublish the card before
editing.");

        card.ClassTeacherComment = request.ClassTeacherComment;
        card.HeadTeacherComment = request.HeadTeacherComment;
        if (request.NextTermBegins.HasValue) card.NextTermBegins = request.NextTermBegins;

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult>
UpsertAffectiveRatingAsync(UpsertAffectiveRatingRequest request, CancellationToken ct =
default)
    {
        var card = await db.ReportCards.FirstOrDefaultAsync(c => c.Id ==
request.ReportCardId, ct);
        if (card is null) return OperationResult.Failure("Report card not found.");
        if (card.IsPublished) return OperationResult.Failure("Unpublish the card before
editing ratings.");

        if (!await db.AffectiveTraits.AnyAsync(t => t.Id == request.AffectiveTraitId, ct))
```

```
            return OperationResult.Failure("Affective trait not found.");

        if (!await db.TraitRatings.AnyAsync(r => r.Id == request.TraitRatingId, ct))
            return OperationResult.Failure("Trait rating not found.");

        var existing = await db.AffectiveRatings.FirstOrDefaultAsync(r =>
            r.ReportCardId == request.ReportCardId
            && r.AffectiveTraitId == request.AffectiveTraitId, ct);

        if (existing is null)
        {
            db.AffectiveRatings.Add(new AffectiveRating
            {
                ReportCardId = request.ReportCardId,
                AffectiveTraitId = request.AffectiveTraitId,
                TraitRatingId = request.TraitRatingId,
            });
        }
        else
        {
            existing.TraitRatingId = request.TraitRatingId;
        }

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult>
UpsertPsychomotorRatingAsync(UpsertPsychomotorRatingRequest request, CancellationToken ct =
default)
    {
        var card = await db.ReportCards.FirstOrDefaultAsync(c => c.Id ==
request.ReportCardId, ct);
        if (card is null) return OperationResult.Failure("Report card not found.");
        if (card.IsPublished) return OperationResult.Failure("Unpublish the card before
editing ratings.");

        if (!await db.PsychomotorSkills.AnyAsync(s => s.Id == request.PsychomotorSkillId,
ct))
            return OperationResult.Failure("Psychomotor skill not found.");

        if (!await db.TraitRatings.AnyAsync(r => r.Id == request.TraitRatingId, ct))
            return OperationResult.Failure("Trait rating not found.");

        var existing = await db.PsychomotorRatings.FirstOrDefaultAsync(r =>
            r.ReportCardId == request.ReportCardId
            && r.PsychomotorSkillId == request.PsychomotorSkillId, ct);

        if (existing is null)
        {
            db.PsychomotorRatings.Add(new PsychomotorRating
            {
                ReportCardId = request.ReportCardId,
                PsychomotorSkillId = request.PsychomotorSkillId,
```

```
                TraitRatingId = request.TraitRatingId,
            });
        }
        else
        {
            existing.TraitRatingId = request.TraitRatingId;
        }

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> PublishAsync(Guid id, CancellationToken ct =
default)
    {
        var card = await db.ReportCards.FirstOrDefaultAsync(c => c.Id == id, ct);
        if (card is null) return OperationResult.Failure("Report card not found.");

        card.IsPublished = true;
        card.PublishedOn = DateTimeOffset.UtcNow;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> UnpublishAsync(Guid id, CancellationToken ct =
default)
    {
        var card = await db.ReportCards.FirstOrDefaultAsync(c => c.Id == id, ct);
        if (card is null) return OperationResult.Failure("Report card not found.");

        card.IsPublished = false;
        card.PublishedOn = null;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var card = await db.ReportCards.FirstOrDefaultAsync(c => c.Id == id, ct);
        if (card is null) return OperationResult.Failure("Report card not found.");

        if (card.IsPublished)
            return OperationResult.Failure("Unpublish the card before deleting.");

        db.ReportCards.Remove(card);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }
}
```

GetByIdAsync and GetForStudentTermAsync share a private BuildDetailAsync that loads the SubjectResults, AffectiveRatings, and PsychomotorRatings as flat DTOs ready for the detail page.

6.4 LookupService — five new methods

All five new methods follow the same pattern: order by DisplayOrder, project to LookupDto, return the list. GetGradeBandsAsync stuffs the description into the Code slot of LookupDto so the UI can show 'A — Excellent' without an extra round-trip.

Excerpt — LookupService.cs (sprint 5 additions)

```
    public async Task<IReadOnlyList<LookupDto>> GetAssessmentTypesAsync(CancellationToken
ct = default) =>
        await db.AssessmentTypes
            .OrderBy(t => t.DisplayOrder)
            .Select(t => new LookupDto { Id = t.Id, Name = t.Name, Code = t.Code })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetGradeBandsAsync(CancellationToken ct =
default) =>
        await db.GradeBands
            .OrderBy(g => g.DisplayOrder)
            .Select(g => new LookupDto { Id = g.Id, Name = g.Name, Code = g.Description })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetAffectiveTraitsAsync(CancellationToken
ct = default) =>
        await db.AffectiveTraits
            .OrderBy(t => t.DisplayOrder)
            .Select(t => new LookupDto { Id = t.Id, Name = t.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetPsychomotorSkillsAsync(CancellationToken
ct = default) =>
        await db.PsychomotorSkills
            .OrderBy(s => s.DisplayOrder)
            .Select(s => new LookupDto { Id = s.Id, Name = s.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetTraitRatingsAsync(CancellationToken ct =
default) =>
```

6.5 DI registration

DependencyInjection.cs picks up three new lines.

Excerpt — DependencyInjection.cs (sprint 5 additions)

```
        services.AddScoped<IAssessmentService, AssessmentService>();
        services.AddScoped<IResultService, ResultService>();
        services.AddScoped<IReportCardService, ReportCardService>();

        return services;
```

7. The EF Core migration

A single migration named AssessmentsAndResults adds eleven new tables (AssessmentTypes, GradeBands, AffectiveTraits, PsychomotorSkills, TraitRatings, TermAssessments, AssessmentScores, SubjectResults, ReportCards, AffectiveRatings, PsychomotorRatings) and the indexes that go with them. It was generated with:

```
dotnet ef migrations add AssessmentsAndResults \
  --project src/NaijaPrimeSchool.Infrastructure \
  --startup-project src/NaijaPrimeSchool.Web \
  --output-dir Persistence/Migrations
```

On a fresh checkout the migration runs at startup via DatabaseInitializer.MigrateAsync. The full Up() method is embedded below for reference.

Excerpt — Up() of 20260504172028_AssessmentsAndResults.cs

```
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "AffectiveTraits",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(60)", maxLength: 60,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_AffectiveTraits", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "AssessmentTypes",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(50)", maxLength: 50,
nullable: false),
                    Code = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
```

```
nullable: false),
                    DefaultMaxScore = table.Column<int>(type: "int", nullable: false),
                    IsExam = table.Column<bool>(type: "bit", nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_AssessmentTypes", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "GradeBands",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(20)", maxLength: 20,
nullable: false),
                    LowerBound = table.Column<decimal>(type: "decimal(5,2)", precision: 5,
scale: 2, nullable: false),
                    UpperBound = table.Column<decimal>(type: "decimal(5,2)", precision: 5,
scale: 2, nullable: false),
                    Description = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: false),
                    Remark = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: true),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
```

```
                    table.PrimaryKey("PK_GradeBands", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "PsychomotorSkills",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(60)", maxLength: 60,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_PsychomotorSkills", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "ReportCards",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    TermId = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    SchoolClassId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    SubjectsTaken = table.Column<int>(type: "int", nullable: false),
                    TotalScore = table.Column<decimal>(type: "decimal(7,2)", precision: 7,
scale: 2, nullable: false),
                    AveragePercentage = table.Column<decimal>(type: "decimal(5,2)",
precision: 5, scale: 2, nullable: false),
                    Position = table.Column<int>(type: "int", nullable: true),
                    StudentsInClass = table.Column<int>(type: "int", nullable: true),
                    DaysPresent = table.Column<int>(type: "int", nullable: false),
                    DaysAbsent = table.Column<int>(type: "int", nullable: false),
                    DaysLate = table.Column<int>(type: "int", nullable: false),
                    TotalSchoolDays = table.Column<int>(type: "int", nullable: false),
                    ClassTeacherComment = table.Column<string>(type: "nvarchar(1000)",
maxLength: 1000, nullable: true),
                    HeadTeacherComment = table.Column<string>(type: "nvarchar(1000)",
maxLength: 1000, nullable: true),
```

```
                    NextTermBegins = table.Column<DateOnly>(type: "date", nullable: true),
                    IsPublished = table.Column<bool>(type: "bit", nullable: false),
                    PublishedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_ReportCards", x => x.Id);
                    table.ForeignKey(
                        name: "FK_ReportCards_SchoolClasses_SchoolClassId",
                        column: x => x.SchoolClassId,
                        principalTable: "SchoolClasses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_ReportCards_Students_StudentId",
                        column: x => x.StudentId,
                        principalTable: "Students",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_ReportCards_Terms_TermId",
                        column: x => x.TermId,
                        principalTable: "Terms",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                });

            migrationBuilder.CreateTable(
                name: "TraitRatings",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
                    Value = table.Column<int>(type: "int", nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
```

```
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_TraitRatings", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "TermAssessments",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    TermId = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    SchoolClassId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    SubjectId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    AssessmentTypeId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    Title = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: false),
                    MaxScore = table.Column<int>(type: "int", nullable: false),
                    Weight = table.Column<decimal>(type: "decimal(5,2)", precision: 5,
scale: 2, nullable: false),
                    AssessmentDate = table.Column<DateOnly>(type: "date", nullable: true),
                    IsPublished = table.Column<bool>(type: "bit", nullable: false),
                    PublishedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    Notes = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_TermAssessments", x => x.Id);
                    table.ForeignKey(
```

```
                        name: "FK_TermAssessments_AssessmentTypes_AssessmentTypeId",
                        column: x => x.AssessmentTypeId,
                        principalTable: "AssessmentTypes",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_TermAssessments_SchoolClasses_SchoolClassId",
                        column: x => x.SchoolClassId,
                        principalTable: "SchoolClasses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_TermAssessments_Subjects_SubjectId",
                        column: x => x.SubjectId,
                        principalTable: "Subjects",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_TermAssessments_Terms_TermId",
                        column: x => x.TermId,
                        principalTable: "Terms",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                });

            migrationBuilder.CreateTable(
                name: "SubjectResults",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    TermId = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    SubjectId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    SchoolClassId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    TotalScore = table.Column<decimal>(type: "decimal(7,2)", precision: 7,
scale: 2, nullable: false),
                    Percentage = table.Column<decimal>(type: "decimal(5,2)", precision: 5,
scale: 2, nullable: false),
                    GradeBandId = table.Column<Guid>(type: "uniqueidentifier", nullable:
true),
                    Position = table.Column<int>(type: "int", nullable: true),
                    StudentsInClass = table.Column<int>(type: "int", nullable: true),
                    TeacherComment = table.Column<string>(type: "nvarchar(500)", maxLength:
500, nullable: true),
                    IsFinalised = table.Column<bool>(type: "bit", nullable: false),
                    FinalisedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
```

```
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_SubjectResults", x => x.Id);
                    table.ForeignKey(
                        name: "FK_SubjectResults_GradeBands_GradeBandId",
                        column: x => x.GradeBandId,
                        principalTable: "GradeBands",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_SubjectResults_SchoolClasses_SchoolClassId",
                        column: x => x.SchoolClassId,
                        principalTable: "SchoolClasses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_SubjectResults_Students_StudentId",
                        column: x => x.StudentId,
                        principalTable: "Students",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_SubjectResults_Subjects_SubjectId",
                        column: x => x.SubjectId,
                        principalTable: "Subjects",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_SubjectResults_Terms_TermId",
                        column: x => x.TermId,
                        principalTable: "Terms",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                });

            migrationBuilder.CreateTable(
                name: "AffectiveRatings",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    ReportCardId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    AffectiveTraitId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    TraitRatingId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
```

```
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_AffectiveRatings", x => x.Id);
                    table.ForeignKey(
                        name: "FK_AffectiveRatings_AffectiveTraits_AffectiveTraitId",
                        column: x => x.AffectiveTraitId,
                        principalTable: "AffectiveTraits",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_AffectiveRatings_ReportCards_ReportCardId",
                        column: x => x.ReportCardId,
                        principalTable: "ReportCards",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                    table.ForeignKey(
                        name: "FK_AffectiveRatings_TraitRatings_TraitRatingId",
                        column: x => x.TraitRatingId,
                        principalTable: "TraitRatings",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                });

            migrationBuilder.CreateTable(
                name: "PsychomotorRatings",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    ReportCardId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    PsychomotorSkillId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    TraitRatingId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
```

```
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_PsychomotorRatings", x => x.Id);
                    table.ForeignKey(
                        name: "FK_PsychomotorRatings_PsychomotorSkills_PsychomotorSkillId",
                        column: x => x.PsychomotorSkillId,
                        principalTable: "PsychomotorSkills",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_PsychomotorRatings_ReportCards_ReportCardId",
                        column: x => x.ReportCardId,
                        principalTable: "ReportCards",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                    table.ForeignKey(
                        name: "FK_PsychomotorRatings_TraitRatings_TraitRatingId",
                        column: x => x.TraitRatingId,
                        principalTable: "TraitRatings",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                });

            migrationBuilder.CreateTable(
                name: "AssessmentScores",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    TermAssessmentId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    Score = table.Column<decimal>(type: "decimal(7,2)", precision: 7,
scale: 2, nullable: true),
                    IsAbsent = table.Column<bool>(type: "bit", nullable: false),
                    Remarks = table.Column<string>(type: "nvarchar(300)", maxLength: 300,
nullable: true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
```

```
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_AssessmentScores", x => x.Id);
                    table.ForeignKey(
                        name: "FK_AssessmentScores_Students_StudentId",
                        column: x => x.StudentId,
                        principalTable: "Students",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_AssessmentScores_TermAssessments_TermAssessmentId",
                        column: x => x.TermAssessmentId,
                        principalTable: "TermAssessments",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                });

            migrationBuilder.CreateIndex(
                name: "IX_AffectiveRatings_AffectiveTraitId",
                table: "AffectiveRatings",
                column: "AffectiveTraitId");

            migrationBuilder.CreateIndex(
                name: "IX_AffectiveRatings_IsDeleted",
                table: "AffectiveRatings",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_AffectiveRatings_ReportCardId_AffectiveTraitId",
                table: "AffectiveRatings",
                columns: new[] { "ReportCardId", "AffectiveTraitId" },
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_AffectiveRatings_TraitRatingId",
                table: "AffectiveRatings",
                column: "TraitRatingId");

            migrationBuilder.CreateIndex(
                name: "IX_AffectiveTraits_Name",
                table: "AffectiveTraits",
                column: "Name",
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_AssessmentScores_IsDeleted",
                table: "AssessmentScores",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_AssessmentScores_StudentId",
```

```
            table: "AssessmentScores",
            column: "StudentId");

        migrationBuilder.CreateIndex(
            name: "IX_AssessmentScores_TermAssessmentId_StudentId",
            table: "AssessmentScores",
            columns: new[] { "TermAssessmentId", "StudentId" },
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_AssessmentTypes_Code",
            table: "AssessmentTypes",
            column: "Code",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_AssessmentTypes_Name",
            table: "AssessmentTypes",
            column: "Name",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_GradeBands_Name",
            table: "GradeBands",
            column: "Name",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_PsychomotorRatings_IsDeleted",
            table: "PsychomotorRatings",
            column: "IsDeleted");

        migrationBuilder.CreateIndex(
            name: "IX_PsychomotorRatings_PsychomotorSkillId",
            table: "PsychomotorRatings",
            column: "PsychomotorSkillId");

        migrationBuilder.CreateIndex(
            name: "IX_PsychomotorRatings_ReportCardId_PsychomotorSkillId",
            table: "PsychomotorRatings",
            columns: new[] { "ReportCardId", "PsychomotorSkillId" },
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_PsychomotorRatings_TraitRatingId",
            table: "PsychomotorRatings",
            column: "TraitRatingId");

        migrationBuilder.CreateIndex(
            name: "IX_PsychomotorSkills_Name",
            table: "PsychomotorSkills",
            column: "Name",
            unique: true);
```

```
            migrationBuilder.CreateIndex(
                name: "IX_ReportCards_IsDeleted",
                table: "ReportCards",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_ReportCards_SchoolClassId_TermId",
                table: "ReportCards",
                columns: new[] { "SchoolClassId", "TermId" });

            migrationBuilder.CreateIndex(
                name: "IX_ReportCards_StudentId_TermId",
                table: "ReportCards",
                columns: new[] { "StudentId", "TermId" },
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_ReportCards_TermId",
                table: "ReportCards",
                column: "TermId");

            migrationBuilder.CreateIndex(
                name: "IX_SubjectResults_GradeBandId",
                table: "SubjectResults",
                column: "GradeBandId");

            migrationBuilder.CreateIndex(
                name: "IX_SubjectResults_IsDeleted",
                table: "SubjectResults",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_SubjectResults_SchoolClassId_TermId_SubjectId",
                table: "SubjectResults",
                columns: new[] { "SchoolClassId", "TermId", "SubjectId" });

            migrationBuilder.CreateIndex(
                name: "IX_SubjectResults_StudentId_TermId_SubjectId",
                table: "SubjectResults",
                columns: new[] { "StudentId", "TermId", "SubjectId" },
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_SubjectResults_SubjectId",
                table: "SubjectResults",
                column: "SubjectId");

            migrationBuilder.CreateIndex(
                name: "IX_SubjectResults_TermId",
                table: "SubjectResults",
                column: "TermId");

            migrationBuilder.CreateIndex(
                name: "IX_TermAssessments_AssessmentTypeId",
```

```
                table: "TermAssessments",
                column: "AssessmentTypeId");

            migrationBuilder.CreateIndex(
                name: "IX_TermAssessments_IsDeleted",
                table: "TermAssessments",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_TermAssessments_SchoolClassId",
                table: "TermAssessments",
                column: "SchoolClassId");

            migrationBuilder.CreateIndex(
                name: "IX_TermAssessments_SubjectId",
                table: "TermAssessments",
                column: "SubjectId");

            migrationBuilder.CreateIndex(
                name: "IX_TermAssessments_TermId_SchoolClassId_SubjectId",
                table: "TermAssessments",
                columns: new[] { "TermId", "SchoolClassId", "SubjectId" });

            migrationBuilder.CreateIndex(
                name: "IX_TraitRatings_Name",
                table: "TraitRatings",
                column: "Name",
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_TraitRatings_Value",
                table: "TraitRatings",
                column: "Value",
                unique: true);
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
```

Sequence: lookup tables first, then TermAssessments (which references AssessmentTypes plus the sprint-2 Term/SchoolClass/Subject), then AssessmentScores (which references TermAssessment and the sprint-3 Student), then SubjectResults and ReportCards (which reference everything else), then the two rating tables. EF Core computes this from the foreign-key graph automatically.

7.1 The schema warning

The migration emits the same long-standing IdentityRole warning every previous sprint has emitted:

```
warn: Microsoft.EntityFrameworkCore.Model.Validation[10622]
      Entity 'ApplicationRole' has a global query filter
      defined and is the required end of a relationship with
      the entity 'ApplicationUserRole'. ...
```

Unrelated to sprint 5. Runtime is fine because we never soft-delete a role from the UI. The role-management UI will revisit this in a later sprint.

8. Seeding the results lookups

DatabaseInitializer picks up a new SeedResultsLookupsAsync method invoked between SeedAttendanceLookupsAsync and SeedRolesAsync. It seeds the five new lookup tables with sensible defaults a Nigerian primary school would want on day one.

Excerpt — SeedResultsLookupsAsync

```
    private static async Task SeedResultsLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
    {
        if (!await db.AssessmentTypes.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, string Code, int Max, bool IsExam)[] types =
            [
                ("First CA",      "CA1",  20, false),
                ("Second CA",     "CA2",  20, false),
                ("Mid-Term Test", "MID",  20, false),
                ("Assignment",    "ASN",  10, false),
                ("Project",       "PRJ",  10, false),
                ("Examination",   "EXAM", 60, true),
            ];
            for (var i = 0; i < types.Length; i++)
            {
                var t = types[i];
                db.AssessmentTypes.Add(new AssessmentType
                {
                    Name = t.Name,
                    Code = t.Code,
                    DefaultMaxScore = t.Max,
                    IsExam = t.IsExam,
                    DisplayOrder = i + 1,
                });
            }
        }

        if (!await db.GradeBands.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, decimal Lo, decimal Hi, string Desc, string Remark)[] bands =
            [
                ("A", 80m, 100m, "Excellent",  "Outstanding performance — keep it up."),
                ("B", 70m, 79.99m,  "Very Good",  "Very good — aim higher."),
                ("C", 60m, 69.99m,  "Good",       "Good — apply more effort."),
                ("D", 50m, 59.99m,  "Pass",       "Fair — work harder."),
                ("E", 40m, 49.99m,  "Weak Pass",  "Below average — needs improvement."),
                ("F",  0m, 39.99m,  "Fail",       "Poor — extra support required."),
            ];
            for (var i = 0; i < bands.Length; i++)
            {
                var g = bands[i];
                db.GradeBands.Add(new GradeBand
                {
                    Name = g.Name,
                    LowerBound = g.Lo,
                    UpperBound = g.Hi,
```

```
                    Description = g.Desc,
                    Remark = g.Remark,
                    DisplayOrder = i + 1,
                });
            }
        }

        if (!await db.AffectiveTraits.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] traits =
            [
                "Punctuality",
                "Attentiveness",
                "Honesty",
                "Neatness",
                "Politeness",
                "Cooperation",
                "Class Participation",
                "Self-control",
            ];
            for (var i = 0; i < traits.Length; i++)
            {
                db.AffectiveTraits.Add(new AffectiveTrait { Name = traits[i], DisplayOrder
= i + 1 });
            }
        }

        if (!await db.PsychomotorSkills.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] skills =
            [
                "Handwriting",
                "Drawing & Painting",
                "Music",
                "Sports & Games",
                "Public Speaking",
                "Crafts",
                "Verbal Fluency",
            ];
            for (var i = 0; i < skills.Length; i++)
            {
                db.PsychomotorSkills.Add(new PsychomotorSkill { Name = skills[i],
DisplayOrder = i + 1 });
            }
        }

        if (!await db.TraitRatings.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, int Value)[] ratings =
            [
                ("Excellent", 5),
                ("Very Good", 4),
                ("Good",      3),
                ("Fair",      2),
                ("Poor",      1),
```

```
            ];
            for (var i = 0; i < ratings.Length; i++)
            {
                db.TraitRatings.Add(new TraitRating
                {
                    Name = ratings[i].Name,
                    Value = ratings[i].Value,
                    DisplayOrder = i + 1,
                });
            }
        }

        await db.SaveChangesAsync(ct);
    }

    private static async Task SeedAttendanceLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
    {
        if (!await db.AttendanceStatuses.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, string Code, bool CountsAsPresent)[] statuses =
            [
                ("Present",   "P",  true),
                ("Late",      "L",  true),
                ("Excused",   "E",  false),
                ("Sick",      "S",  false),
                ("Absent",    "A",  false),
                ("Suspended", "SP", false),
            ];
            for (var i = 0; i < statuses.Length; i++)
            {
                db.AttendanceStatuses.Add(new AttendanceStatus
                {
                    Name = statuses[i].Name,
                    Code = statuses[i].Code,
                    DisplayOrder = i + 1,
                    CountsAsPresent = statuses[i].CountsAsPresent,
                });
            }
        }

        await db.SaveChangesAsync(ct);
    }

    private static async Task SeedFamilyLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
    {
        if (!await db.Relationships.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] relationships =
            [
                "Father",
                "Mother",
                "Stepfather",
```

```
                "Stepmother",
                "Grandfather",
                "Grandmother",
                "Uncle",
                "Aunt",
                "Guardian",
                "Other",
            ];
            for (var i = 0; i < relationships.Length; i++)
            {
                db.Relationships.Add(new Relationship { Name = relationships[i],
DisplayOrder = i + 1 });
            }
        }

        if (!await db.EnrolmentStatuses.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] statuses =
            [
                "Active",
                "Suspended",
                "Withdrawn",
                "Transferred",
                "Graduated",
            ];
            for (var i = 0; i < statuses.Length; i++)
            {
                db.EnrolmentStatuses.Add(new EnrolmentStatus { Name = statuses[i],
DisplayOrder = i + 1 });
            }
        }

        if (!await db.BloodGroups.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] groups = ["A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-",
"Unknown"];
            for (var i = 0; i < groups.Length; i++)
            {
                db.BloodGroups.Add(new BloodGroup { Name = groups[i], DisplayOrder = i +
1 });
            }
        }

        if (!await db.MaritalStatuses.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] statuses = ["Single", "Married", "Divorced", "Widowed", "Separated"];
            for (var i = 0; i < statuses.Length; i++)
            {
                db.MaritalStatuses.Add(new MaritalStatus { Name = statuses[i], DisplayOrder
= i + 1 });
            }
        }

        await db.SaveChangesAsync(ct);
```

```
    }

    private static async Task SeedAcademicLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
```

What gets seeded:

- AssessmentTypes — First CA, Second CA, Mid-Term Test, Assignment, Project, Examination (6 rows). Examination is the one with IsExam = true so the UI can flag it.
- GradeBands — A (80–100), B (70–79.99), C (60–69.99), D (50–59.99), E (40–49.99), F (0–39.99). Each band carries a Description and a Remark, and the bounds use decimal(5,2) for precision.
- AffectiveTraits — Punctuality, Attentiveness, Honesty, Neatness, Politeness, Cooperation, Class Participation, Self-control (8 rows).
- PsychomotorSkills — Handwriting, Drawing & Painting, Music, Sports & Games, Public Speaking, Crafts, Verbal Fluency (7 rows).
- TraitRatings — Excellent (5), Very Good (4), Good (3), Fair (2), Poor (1).

Same .IgnoreQueryFilters().AnyAsync() guard pattern as earlier sprints — the seeder only inserts if the table is empty (counting soft-deleted rows), so running the app a second time after a soft delete does not resurrect the row.

9. The Razor pages

Four new pages land in src/NaijaPrimeSchool.Web/Components/Pages/Results/. The pattern mirrors earlier sprints: a list page (with optional inline form), a detail page (for the score sheet or the report card detail), a per-domain action page (Results page, ReportCards page).

9.1 Page roster

```
src/NaijaPrimeSchool.Web/Components/Pages/Results/
├── Assessments.razor          <- /assessments
├── AssessmentScores.razor     <- /assessments/{id}/scores
├── Results.razor              <- /results
├── ReportCards.razor          <- /reports
└── ReportCardDetail.razor     <- /reports/{id}
```

Assessment pages are gated to SuperAdmin + HeadTeacher + Teacher (a class teacher needs to enter scores). Results and ReportCards are SuperAdmin + HeadTeacher only — those are school-wide computations that we do not want a single class teacher kicking off accidentally.

9.2 Assessments.razor — the gradebook list

Search by session/term/class/subject, paged grid showing each assessment with its scored count, status badge, and a row of actions (open score sheet, edit, publish/unpublish, delete). The new-assessment form is inline below the grid; once an assessment exists, its term/class/subject are locked because moving an assessment to a different (class, subject) would invalidate scores.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Results/Assessments.razor

```
@page "/assessments"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},{Roles.Teacher}")]
@rendermode InteractiveServer

@inject IAssessmentService AssessmentService
@inject ISessionService SessionService
@inject ITermService TermService
@inject ISubjectService SubjectService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Assessments · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Assessments</h1>
        <p>Continuous assessments, mid-term tests, projects and exams. Set up the gradebook
here, then enter scores from each row.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="New assessment" Icon="add" ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => OpenForm(null))" />
    </div>
</div>
```

```
<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Session</label>
            <RadzenDropDown TValue="Guid?" Data="@sessions" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="sessionId" AllowClear="true" Placeholder="All
sessions"
                            Change="@(_ => SessionChangedAsync())" Style="min-width:
200px;" />
        </div>
        <div class="nps-field">
            <label>Term</label>
            <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
                            AllowClear="true" Placeholder="All terms"
                            Change="@(_ => LoadAsync())" Style="min-width: 200px;">
                <Template Context="t">
                    @{ var dto = (TermDto)t; }
                    @dto.TermTypeName — @dto.SessionName
                </Template>
            </RadzenDropDown>
        </div>
        <div class="nps-field">
            <label>Class</label>
            <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="classId" AllowClear="true" Placeholder="All
classes"
                            Change="@(_ => LoadAsync())" Style="min-width: 240px;" />
        </div>
        <div class="nps-field">
            <label>Subject</label>
            <RadzenDropDown TValue="Guid?" Data="@subjects" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="subjectId" AllowClear="true" Placeholder="All
subjects"
                            Change="@(_ => LoadAsync())" Style="min-width: 200px;" />
        </div>
        <RadzenButton Text="Refresh" Icon="refresh" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@LoadAsync" />
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="TermAssessmentDto" Data="@assessments" IsLoading="@loading"
                    AllowPaging="true" PageSize="25" EmptyText="No assessments match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Title" Property="Title">
                <Template Context="a">
                    <strong>@a.Title</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">
                        @a.AssessmentTypeName · max @a.MaxScore · weight @a.Weight
```

```
                    </small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Class & subject">
                <Template Context="a">
                    <strong>@a.SchoolClassName</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">@a.SubjectName
(@a.SubjectCode)</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Term"
Property="TermName" Width="200px" />
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Date"
Property="AssessmentDate"
                                  FormatString="{0:d MMM yyyy}" Width="140px">
                <Template Context="a">
                    @(a.AssessmentDate.HasValue ? a.AssessmentDate.Value.ToString("d MMM
yyyy") : "—")
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Scored" Width="100px">
                <Template Context="a">
                    <span>@a.ScoredCount / @a.ExpectedCount</span>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Status" Width="120px">
                <Template Context="a">
                    @if (a.IsPublished)
                    {
                        <RadzenBadge Text="Published" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="Draft" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="TermAssessmentDto" Title="Actions"
Sortable="false"
                                  Width="220px" TextAlign="TextAlign.Right">
                <Template Context="a">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="edit_note" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Primary" title="Enter
scores"
                                      Click="@(() =>
Nav.NavigateTo($"/assessments/{a.Id}/scores"))" />
                        <RadzenButton Icon="edit" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Light" title="Edit"
                                      Click="@(() => OpenForm(a))" />
```

```
                        @if (a.IsPublished)
                        {
                            <RadzenButton Icon="lock_open" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Warning"
title="Unpublish"
                                          Click="@(() => UnpublishAsync(a))" />
                        }
                        else
                        {
                            <RadzenButton Icon="lock" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Success" title="Publish"
                                          Click="@(() => PublishAsync(a))" />
                        }
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Danger" title="Delete"
                                      Click="@(() => DeleteAsync(a))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@if (showForm)
{
    <RadzenCard class="nps-card" Style="margin-top:1rem;">
        <h3 style="margin:0 0 1rem;">@(editingId.HasValue ? "Edit assessment" : "New
assessment")</h3>
        <RadzenTemplateForm TItem="AssessmentFormModel" Data="@form" Submit="@SaveAsync">
            <DataAnnotationsValidator />
            <div class="nps-form-grid">
                <div class="nps-field">
                    <label>Term *</label>
                    <RadzenDropDown TValue="Guid?" Data="@termsForForm" @bind-
Value="form.TermId"
                                    Placeholder="Pick a term"
Disabled="@editingId.HasValue">
                        <Template Context="t">
                            @{ var dto = (TermDto)t; }
                            @dto.TermTypeName — @dto.SessionName
                        </Template>
                    </RadzenDropDown>
                </div>
                <div class="nps-field">
                    <label>Class *</label>
                    <RadzenDropDown TValue="Guid?" Data="@classesForForm"
TextProperty="Name" ValueProperty="Id"
                                    @bind-Value="form.SchoolClassId" Placeholder="Pick a
class"
                                    Disabled="@editingId.HasValue" />
                </div>
                <div class="nps-field">
```

```
                    <label>Subject *</label>
                    <RadzenDropDown TValue="Guid?" Data="@subjects" TextProperty="Name"
ValueProperty="Id"
                                    @bind-Value="form.SubjectId" Placeholder="Pick a
subject"
                                    Disabled="@editingId.HasValue" />
                </div>
                <div class="nps-field">
                    <label>Type *</label>
                    <RadzenDropDown TValue="Guid?" Data="@assessmentTypes"
TextProperty="Name" ValueProperty="Id"
                                    @bind-Value="form.AssessmentTypeId" Placeholder="Pick a
type" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Title *</label>
                    <RadzenTextBox @bind-Value="form.Title" Placeholder="e.g. Week 6 test
on fractions" />
                    <ValidationMessage For="() => form.Title" />
                </div>
                <div class="nps-field">
                    <label>Maximum score *</label>
                    <RadzenNumeric TValue="int" @bind-Value="form.MaxScore" Min="1"
Max="1000" />
                </div>
                <div class="nps-field">
                    <label>Weight</label>
                    <RadzenNumeric TValue="decimal" @bind-Value="form.Weight" Min="0"
Step="0.1" />
                    <small style="color: var(--nps-ink-500);">Multiplier when computing the
term's subject total.</small>
                </div>
                <div class="nps-field">
                    <label>Date</label>
                    <RadzenDatePicker TValue="DateTime?" @bind-Value="form.AssessmentDate"
                                      DateFormat="d MMM yyyy" ShowTime="false" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Notes</label>
                    <RadzenTextArea @bind-Value="form.Notes" Rows="2" />
                </div>
            </div>
            <div class="nps-form-actions">
                <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                              ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                              Click="@(() => showForm = false)" />
                <RadzenButton Text="@(saving ? "Saving..." : "Save")"
                              Icon="save" ButtonType="ButtonType.Submit"
                              ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
            </div>
        </RadzenTemplateForm>
    </RadzenCard>
}

@code {
```

```
    private IReadOnlyList<TermAssessmentDto> assessments = [];
    private IReadOnlyList<SessionDto> sessions = [];
    private List<TermDto> terms = [];
    private List<TermDto> termsForForm = [];
    private IReadOnlyList<LookupDto> classes = [];
    private IReadOnlyList<LookupDto> classesForForm = [];
    private IReadOnlyList<SubjectDto> subjects = [];
    private IReadOnlyList<LookupDto> assessmentTypes = [];

    private bool loading;
    private bool saving;
    private bool showForm;
    private Guid? editingId;

    private Guid? sessionId;
    private Guid? termId;
    private Guid? classId;
    private Guid? subjectId;

    private AssessmentFormModel form = new();

    private class AssessmentFormModel
    {
        public Guid? TermId { get; set; }
        public Guid? SchoolClassId { get; set; }
        public Guid? SubjectId { get; set; }
        public Guid? AssessmentTypeId { get; set; }

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(120)]
        public string Title { get; set; } = string.Empty;

        public int MaxScore { get; set; } = 100;
        public decimal Weight { get; set; } = 1m;
        public DateTime? AssessmentDate { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(500)]
        public string? Notes { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        sessions = await SessionService.ListAsync();
        sessionId = sessions.FirstOrDefault(s => s.IsCurrent)?.Id;
        await ReloadTermsAsync();
        termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id ?? terms.FirstOrDefault()?.Id;
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
        classesForForm = await LookupService.GetClassesForSessionAsync(sessionId);
        subjects = await SubjectService.ListAsync();
        assessmentTypes = await LookupService.GetAssessmentTypesAsync();
        await LoadAsync();
    }

    private async Task ReloadTermsAsync()
```

```
    {
        var allTerms = await TermService.ListAsync();
        terms = allTerms
            .Where(t => !sessionId.HasValue || t.SessionId == sessionId.Value)
            .ToList();
        termsForForm = allTerms.ToList();
    }

    private async Task SessionChangedAsync()
    {
        termId = null;
        classId = null;
        await ReloadTermsAsync();
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            assessments = await AssessmentService.ListAsync(new TermAssessmentFilter
            {
                TermId = termId,
                SchoolClassId = classId,
                SubjectId = subjectId,
            });
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private async Task OpenForm(TermAssessmentDto? a)
    {
        editingId = a?.Id;
        if (a is null)
        {
            // Default the form to the current filters when starting a new assessment.
            var defaultType = assessmentTypes.FirstOrDefault();
            form = new AssessmentFormModel
            {
                TermId = termId ?? terms.FirstOrDefault()?.Id,
                SchoolClassId = classId,
                SubjectId = subjectId,
                AssessmentTypeId = defaultType?.Id,
                Title = string.Empty,
                MaxScore = 20,
                Weight = 1m,
                AssessmentDate = DateTime.Today,
            };
        }
```

```
        else
        {
            form = new AssessmentFormModel
            {
                TermId = a.TermId,
                SchoolClassId = a.SchoolClassId,
                SubjectId = a.SubjectId,
                AssessmentTypeId = a.AssessmentTypeId,
                Title = a.Title,
                MaxScore = a.MaxScore,
                Weight = a.Weight,
                AssessmentDate = a.AssessmentDate?.ToDateTime(TimeOnly.MinValue),
                Notes = a.Notes,
            };
        }
        showForm = true;
        await Task.CompletedTask;
    }

    private async Task SaveAsync()
    {
        if (form.TermId is null || form.SchoolClassId is null
            || form.SubjectId is null || form.AssessmentTypeId is null)
        {
            Notification.Notify(NotificationSeverity.Warning, "Missing fields",
                "Pick a term, class, subject and assessment type.");
            return;
        }

        saving = true;
        try
        {
            OperationResult result = editingId.HasValue
                ? await AssessmentService.UpdateAsync(new UpdateTermAssessmentRequest
                {
                    Id = editingId.Value,
                    AssessmentTypeId = form.AssessmentTypeId.Value,
                    Title = form.Title,
                    MaxScore = form.MaxScore,
                    Weight = form.Weight,
                    AssessmentDate = form.AssessmentDate.HasValue
                        ? DateOnly.FromDateTime(form.AssessmentDate.Value) : null,
                    Notes = form.Notes,
                })
                : await AssessmentService.CreateAsync(new CreateTermAssessmentRequest
                {
                    TermId = form.TermId.Value,
                    SchoolClassId = form.SchoolClassId.Value,
                    SubjectId = form.SubjectId.Value,
                    AssessmentTypeId = form.AssessmentTypeId.Value,
                    Title = form.Title,
                    MaxScore = form.MaxScore,
                    Weight = form.Weight,
                    AssessmentDate = form.AssessmentDate.HasValue
                        ? DateOnly.FromDateTime(form.AssessmentDate.Value) : null,
```

```
                    Notes = form.Notes,
                });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved", $"Assessment
'{form.Title}' saved.");
                showForm = false;
                await LoadAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }

    private async Task PublishAsync(TermAssessmentDto a)
    {
        var result = await AssessmentService.PublishAsync(a.Id);
        ShowResult(result, $"'{a.Title}' published.");
        await LoadAsync();
    }

    private async Task UnpublishAsync(TermAssessmentDto a)
    {
        var result = await AssessmentService.UnpublishAsync(a.Id);
        ShowResult(result, $"'{a.Title}' unpublished.");
        await LoadAsync();
    }

    private async Task DeleteAsync(TermAssessmentDto a)
    {
        var confirm = await Dialog.Confirm(
            $"Delete assessment '{a.Title}'?",
            "Delete assessment",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await AssessmentService.SoftDeleteAsync(a.Id);
        ShowResult(result, $"'{a.Title}' deleted.");
        await LoadAsync();
    }

    private void ShowResult(OperationResult result, string success)
    {
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Done", success);
```

```
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
}
```

9.3 AssessmentScores.razor — the score sheet

The score-entry workhorse. Renders an HTML table (.nps-score-grid) — one row per actively-enrolled pupil, columns for score, absent, and remarks. Score numerics are bounded by the assessment's MaxScore. Marking absent clears the score; entering a score clears absent. A single Save Scores button posts the whole sheet via BulkSetScoresAsync. Once the assessment is published, every input on the page is disabled.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Results/AssessmentScores.razor

```
@page "/assessments/{Id:guid}/scores"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},{Roles.Teacher}")]
@rendermode InteractiveServer

@inject IAssessmentService AssessmentService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Score sheet · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading score sheet...</p>
}
else if (sheet is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Assessment not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div>
            <h1>@sheet.Assessment.Title</h1>
            <p>
                @sheet.Assessment.SchoolClassName · @sheet.Assessment.SubjectName
                (@sheet.Assessment.SubjectCode) · @sheet.Assessment.AssessmentTypeName ·
                max <strong>@sheet.Assessment.MaxScore</strong>
            </p>
        </div>
        <div class="nps-page-actions">
            @if (sheet.Assessment.IsPublished)
            {
                <RadzenBadge Text="Published — read only" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
```

```
            }
            else
            {
                <RadzenButton Text="@(saving ? "Saving..." : "Save scores")"
                              Icon="save" ButtonStyle="ButtonStyle.Primary"
                              Disabled="@saving" Click="@SaveAllAsync" />
            }
            <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                          ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/assessments"))" />
        </div>
    </div>

    <RadzenCard class="nps-card">
        @if (sheet.Scores.Count == 0)
        {
            <p style="color: var(--nps-ink-500);">
                No pupils are currently enrolled in
<strong>@sheet.Assessment.SchoolClassName</strong>.
                Enrol pupils first under Family → Students.
            </p>
        }
        else
        {
            <table class="nps-score-grid">
                <thead>
                    <tr>
                        <th style="width: 40px;">#</th>
                        <th>Pupil</th>
                        <th style="width: 120px;">Score</th>
                        <th style="width: 100px;">Absent</th>
                        <th>Remarks</th>
                    </tr>
                </thead>
                <tbody>
                    @for (var i = 0; i < sheet.Scores.Count; i++)
                    {
                        var idx = i;
                        var row = sheet.Scores[idx];
                        <tr>
                            <td>@(idx + 1)</td>
                            <td>
                                <strong>@row.StudentName</strong>
                                <br />
                                <small style="color: var(--nps-
ink-500);">@row.StudentAdmissionNumber</small>
                            </td>
                            <td>
                                <RadzenNumeric TValue="decimal?" Value="row.Score"
                                               Min="0"
Max="@((decimal)sheet.Assessment.MaxScore)"
                                               Step="0.5"
                                               Disabled="@(sheet.Assessment.IsPublished ||
row.IsAbsent)"
                                               Change="@((decimal? v) => UpdateScore(idx,
```

```
v))" />
                            </td>
                            <td style="text-align: center;">
                                <RadzenCheckBox TValue="bool" Value="row.IsAbsent"
                                                Disabled="@sheet.Assessment.IsPublished"
                                                Change="@((bool v) => UpdateAbsent(idx,
v))" />
                            </td>
                            <td>
                                <RadzenTextBox Value="row.Remarks ?? string.Empty"
                                               Disabled="@sheet.Assessment.IsPublished"
                                               Change="@((string v) => UpdateRemarks(idx,
v))"
                                               Style="width: 100%;" />
                            </td>
                        </tr>
                    }
                </tbody>
            </table>
        }
    </RadzenCard>
}

@code {
    [Parameter] public Guid Id { get; set; }

    private AssessmentScoreSheetDto? sheet;
    private bool loading = true;
    private bool saving;

    protected override Task OnInitializedAsync() => LoadAsync();

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            sheet = await AssessmentService.GetScoreSheetAsync(Id);
        }
        finally
        {
            loading = false;
        }
    }

    private void UpdateScore(int index, decimal? value)
    {
        if (sheet is null) return;
        var row = sheet.Scores[index];
        row.Score = value;
        if (value.HasValue) row.IsAbsent = false;
    }

    private void UpdateAbsent(int index, bool value)
```

```
    {
        if (sheet is null) return;
        var row = sheet.Scores[index];
        row.IsAbsent = value;
        if (value) row.Score = null;
    }

    private void UpdateRemarks(int index, string value)
    {
        if (sheet is null) return;
        sheet.Scores[index].Remarks = string.IsNullOrWhiteSpace(value) ? null : value;
    }

    private async Task SaveAllAsync()
    {
        if (sheet is null) return;
        saving = true;
        try
        {
            var rows = sheet.Scores
                .Where(s => s.Score.HasValue || s.IsAbsent || !
string.IsNullOrWhiteSpace(s.Remarks))
                .Select(s => new UpsertAssessmentScoreRequest
                {
                    Id = s.Id == Guid.Empty ? null : s.Id,
                    TermAssessmentId = Id,
                    StudentId = s.StudentId,
                    Score = s.Score,
                    IsAbsent = s.IsAbsent,
                    Remarks = s.Remarks,
                })
                .ToList();

            var result = await AssessmentService.BulkSetScoresAsync(new
BulkSetScoresRequest
            {
                TermAssessmentId = Id,
                Scores = rows,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved",
                    $"{rows.Count} score(s) saved.");
                await LoadAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
            }
        }
        finally
        {
```

```
            saving = false;
        }
    }
}
```

9.4 Results.razor — compute and view

Pick a term and class, optionally narrow to a subject, then either Recompute (idempotent and safe) or Compute & finalise (locks the rows). The grid renders the SubjectResults with grade badges, position counters, and per-row finalise/reopen/delete actions. The colour-coded grade badges use the Radzen BadgeStyle palette so A/B render green/blue, C/D primary/yellow, E/F warning/red.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Results/Results.razor

```
@page "/results"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IResultService ResultService
@inject ISessionService SessionService
@inject ITermService TermService
@inject ISubjectService SubjectService
@inject ILookupService LookupService
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Results · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Subject results</h1>
        <p>Computed from the assessments. Recompute as scores change, finalise when
ready.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Recompute" Icon="calculate" ButtonStyle="ButtonStyle.Info"
                      Click="@(() => ComputeAsync(false))" Disabled="@(!CanCompute)" />
        <RadzenButton Text="Compute & finalise" Icon="lock"
ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => ComputeAsync(true))" Disabled="@(!CanCompute)" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Term *</label>
            <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
                            AllowClear="true" Placeholder="Pick a term"
                            Change="@(_ => LoadAsync())" Style="min-width: 220px;">
                <Template Context="t">
                    @{ var dto = (TermDto)t; }
                    @dto.TermTypeName — @dto.SessionName
                </Template>
```

```
            </RadzenDropDown>
        </div>
        <div class="nps-field">
            <label>Class *</label>
            <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="classId" AllowClear="true" Placeholder="Pick a
class"
                            Change="@(_ => LoadAsync())" Style="min-width: 240px;" />
        </div>
        <div class="nps-field">
            <label>Subject (optional)</label>
            <RadzenDropDown TValue="Guid?" Data="@subjects" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="subjectId" AllowClear="true" Placeholder="All
subjects"
                            Change="@(_ => LoadAsync())" Style="min-width: 200px;" />
        </div>
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    @if (results.Count == 0)
    {
        <p style="color: var(--nps-ink-500);">
            No computed results yet. Pick a term and class above, then click
<strong>Recompute</strong>.
        </p>
    }
    else
    {
        <RadzenDataGrid TItem="SubjectResultDto" Data="@results" IsLoading="@loading"
                        AllowPaging="true" PageSize="50" AllowSorting="true">
            <Columns>
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Pupil">
                    <Template Context="r">
                        <strong>@r.StudentName</strong>
                        <br />
                        <small style="color: var(--nps-
ink-500);">@r.StudentAdmissionNumber</small>
                    </Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Subject"
Property="SubjectName" />
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Total"
Property="TotalScore" Width="100px"
                                      FormatString="{0:F2}" />
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="%"
Property="Percentage" Width="100px">
                    <Template Context="r">
                        <strong>@r.Percentage.ToString("F2")%</strong>
                    </Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Grade" Width="120px">
                    <Template Context="r">
```

```
                        @if (!string.IsNullOrEmpty(r.GradeBandName))
                        {
                            <RadzenBadge Text="@r.GradeBandName"
                                         BadgeStyle="@GradeStyle(r.GradeBandName)"
                                         Variant="Variant.Flat" />
                        }
                    </Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Position"
Width="100px">
                    <Template Context="r">
                        @(r.Position.HasValue ? $"{r.Position} of {r.StudentsInClass}" :
"—")
                    </Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Status"
Width="120px">
                    <Template Context="r">
                        @if (r.IsFinalised)
                        {
                            <RadzenBadge Text="Finalised" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                        }
                        else
                        {
                            <RadzenBadge Text="Draft" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                        }
                    </Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="SubjectResultDto" Title="Actions"
Sortable="false"
                                      Width="180px" TextAlign="TextAlign.Right">
                    <Template Context="r">
                        <div class="nps-inline-actions">
                            @if (r.IsFinalised)
                            {
                                <RadzenButton Icon="lock_open" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                              ButtonStyle="ButtonStyle.Warning"
title="Reopen"
                                              Click="@(() => ReopenAsync(r))" />
                            }
                            else
                            {
                                <RadzenButton Icon="lock" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                              ButtonStyle="ButtonStyle.Success"
title="Finalise"
                                              Click="@(() => FinaliseAsync(r))" />
                            }
                            <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Danger" title="Delete"
                                          Click="@(() => DeleteAsync(r))" />
```

```
                        </div>
                    </Template>
                </RadzenDataGridColumn>
            </Columns>
        </RadzenDataGrid>
    }
</RadzenCard>

@code {
    private List<SubjectResultDto> results = [];
    private List<TermDto> terms = [];
    private IReadOnlyList<LookupDto> classes = [];
    private IReadOnlyList<SubjectDto> subjects = [];

    private bool loading;
    private Guid? termId;
    private Guid? classId;
    private Guid? subjectId;

    private bool CanCompute => termId.HasValue && classId.HasValue;

    protected override async Task OnInitializedAsync()
    {
        terms = (await TermService.ListAsync()).ToList();
        termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id;
        var sessionId = terms.FirstOrDefault(t => t.Id == termId)?.SessionId;
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
        subjects = await SubjectService.ListAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            results = (await ResultService.ListAsync(new SubjectResultFilter
            {
                TermId = termId,
                SchoolClassId = classId,
                SubjectId = subjectId,
            })).ToList();
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private async Task ComputeAsync(bool finalise)
    {
        if (!CanCompute) return;
```

```
        if (finalise)
        {
            var confirm = await Dialog.Confirm(
                "Compute and finalise the results? Once finalised, scores cannot be edited
until you reopen each result.",
                "Finalise results",
                new ConfirmOptions { OkButtonText = "Finalise", CancelButtonText = "Cancel"
});
            if (confirm != true) return;
        }

        var op = await ResultService.ComputeAsync(new ComputeResultsRequest
        {
            TermId = termId!.Value,
            SchoolClassId = classId!.Value,
            SubjectId = subjectId,
            Finalise = finalise,
        });

        if (op.Succeeded && op.Data is not null)
        {
            var msg = $"{op.Data.RowsComputed} row(s) computed";
            if (finalise) msg += $", {op.Data.RowsFinalised} finalised";
            if (op.Data.Warnings.Count > 0) msg += $" ({op.Data.Warnings.Count}
warning(s))";
            Notification.Notify(NotificationSeverity.Success, "Computed", msg);
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
op.Errors));
        }
    }

    private async Task FinaliseAsync(SubjectResultDto r)
    {
        var result = await ResultService.FinaliseAsync(r.Id);
        ShowResult(result, "Result finalised.");
        await LoadAsync();
    }

    private async Task ReopenAsync(SubjectResultDto r)
    {
        var result = await ResultService.ReopenAsync(r.Id);
        ShowResult(result, "Result reopened.");
        await LoadAsync();
    }

    private async Task DeleteAsync(SubjectResultDto r)
    {
        var confirm = await Dialog.Confirm(
            $"Delete the result for {r.StudentName} in {r.SubjectName}?",
            "Delete result",
```

```
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await ResultService.SoftDeleteAsync(r.Id);
        ShowResult(result, "Result deleted.");
        await LoadAsync();
    }

    private void ShowResult(OperationResult result, string success)
    {
        if (result.Succeeded)
            Notification.Notify(NotificationSeverity.Success, "Done", success);
        else
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
    }

    private static BadgeStyle GradeStyle(string? grade) => grade switch
    {
        "A" => BadgeStyle.Success,
        "B" => BadgeStyle.Info,
        "C" => BadgeStyle.Primary,
        "D" => BadgeStyle.Warning,
        "E" => BadgeStyle.Warning,
        "F" => BadgeStyle.Danger,
        _ => BadgeStyle.Secondary,
    };
}
```

9.5 ReportCards.razor — generate and list

Pick a (term, class), see every report card already generated for that pair (with the average, position, attendance summary, and publishing status), or click Generate / refresh to roll a fresh batch. The generate dialog is inline below the grid and lets the head teacher set 'next term begins' once for the whole batch.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Results/ReportCards.razor

```
@page "/reports"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IReportCardService ReportCardService
@inject ITermService TermService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Report cards · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Report cards</h1>
        <p>Generate, edit, and publish per-pupil end-of-term reports.</p>
```

```
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Generate / refresh" Icon="auto_awesome"
ButtonStyle="ButtonStyle.Primary"
                      Click="@OpenGenerateDialog" Disabled="@(!CanGenerate)" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Term</label>
            <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
                            AllowClear="true" Placeholder="All terms"
                            Change="@(_ => LoadAsync())" Style="min-width: 220px;">
                <Template Context="t">
                    @{ var dto = (TermDto)t; }
                    @dto.TermTypeName — @dto.SessionName
                </Template>
            </RadzenDropDown>
        </div>
        <div class="nps-field">
            <label>Class</label>
            <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="classId" AllowClear="true" Placeholder="All
classes"
                            Change="@(_ => LoadAsync())" Style="min-width: 240px;" />
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="bool?" Data="@statusOptions" TextProperty="Text"
ValueProperty="Value"
                            @bind-Value="isPublished" AllowClear="true" Placeholder="All"
                            Change="@(_ => LoadAsync())" Style="min-width: 180px;" />
        </div>
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="ReportCardDto" Data="@cards" IsLoading="@loading"
                    AllowPaging="true" PageSize="50" EmptyText="No report cards yet —
generate them above.">
        <Columns>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Pupil">
                <Template Context="c">
                    <strong>@c.StudentName</strong>
                    <br />
                    <small style="color:
var(--nps-ink-500);">@c.StudentAdmissionNumber</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Term & class">
                <Template Context="c">
                    <strong>@c.SchoolClassName</strong>
```

```
                    <br />
                    <small style="color: var(--nps-ink-500);">@c.TermName</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Subjects"
Property="SubjectsTaken" Width="100px" />
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Average" Width="120px">
                <Template Context="c">
                    <strong>@c.AveragePercentage.ToString("F2")%</strong>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Position" Width="120px">
                <Template Context="c">
                    @(c.Position.HasValue ? $"{c.Position} of {c.StudentsInClass}" : "—")
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Attendance" Width="160px">
                <Template Context="c">
                    @c.DaysPresent / @c.TotalSchoolDays days
                    <br />
                    <small style="color: var(--nps-ink-500);">@c.DaysAbsent absent ·
@c.DaysLate late</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Status" Width="120px">
                <Template Context="c">
                    @if (c.IsPublished)
                    {
                        <RadzenBadge Text="Published" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="Draft" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ReportCardDto" Title="Actions" Sortable="false"
                                  Width="200px" TextAlign="TextAlign.Right">
                <Template Context="c">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="visibility" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Primary" title="Open"
                                      Click="@(() =>
Nav.NavigateTo($"/reports/{c.Id}"))" />
                        @if (c.IsPublished)
                        {
                            <RadzenButton Icon="lock_open" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Warning"
title="Unpublish"
                                          Click="@(() => UnpublishAsync(c))" />
                        }
```

```
                        else
                        {
                            <RadzenButton Icon="lock" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Success" title="Publish"
                                          Click="@(() => PublishAsync(c))" />
                        }
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Danger" title="Delete"
                                      Click="@(() => DeleteAsync(c))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@if (showGenerate)
{
    <RadzenCard class="nps-card" Style="margin-top:1rem;">
        <h3 style="margin: 0 0 1rem;">Generate / refresh report cards</h3>
        <p style="color: var(--nps-ink-500);">
            Pulls together the subject results, attendance counts, and class positions for
every pupil in
            the selected (term, class). Existing draft cards are refreshed; published cards
are left alone.
        </p>
        <div class="nps-form-grid">
            <div class="nps-field">
                <label>Term *</label>
                <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="genTermId"
                                Placeholder="Pick a term">
                    <Template Context="t">
                        @{ var dto = (TermDto)t; }
                        @dto.TermTypeName — @dto.SessionName
                    </Template>
                </RadzenDropDown>
            </div>
            <div class="nps-field">
                <label>Class *</label>
                <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="genClassId" Placeholder="Pick a class" />
            </div>
            <div class="nps-field nps-span-2">
                <label>Next term begins (optional)</label>
                <RadzenDatePicker TValue="DateTime?" @bind-Value="genNextTerm"
                                  DateFormat="d MMM yyyy" ShowTime="false" />
            </div>
        </div>
        <div class="nps-form-actions">
            <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                          ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                          Click="@(() => showGenerate = false)" />
```

```
            <RadzenButton Text="@(generating ? "Generating..." : "Generate")"
                          Icon="auto_awesome" ButtonStyle="ButtonStyle.Primary"
                          Disabled="@generating" Click="@GenerateAsync" />
        </div>
    </RadzenCard>
}

@code {
    private List<ReportCardDto> cards = [];
    private List<TermDto> terms = [];
    private IReadOnlyList<LookupDto> classes = [];

    private bool loading;
    private Guid? termId;
    private Guid? classId;
    private bool? isPublished;

    private record StatusOption(string Text, bool? Value);
    private readonly List<StatusOption> statusOptions =
    [
        new StatusOption("Draft", false),
        new StatusOption("Published", true),
    ];

    private bool showGenerate;
    private bool generating;
    private Guid? genTermId;
    private Guid? genClassId;
    private DateTime? genNextTerm;

    private bool CanGenerate => terms.Count > 0;

    protected override async Task OnInitializedAsync()
    {
        terms = (await TermService.ListAsync()).ToList();
        termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id;
        classes = await LookupService.GetClassesForSessionAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            cards = (await ReportCardService.ListAsync(new ReportCardFilter
            {
                TermId = termId,
                SchoolClassId = classId,
                IsPublished = isPublished,
            })).ToList();
        }
        finally
        {
```

```
            loading = false;
            StateHasChanged();
        }
    }

    private void OpenGenerateDialog()
    {
        genTermId = termId ?? terms.FirstOrDefault(t => t.IsCurrent)?.Id;
        genClassId = classId;
        genNextTerm = null;
        showGenerate = true;
    }

    private async Task GenerateAsync()
    {
        if (genTermId is null || genClassId is null)
        {
            Notification.Notify(NotificationSeverity.Warning, "Missing fields", "Pick a
term and a class.");
            return;
        }

        generating = true;
        try
        {
            var op = await ReportCardService.GenerateAsync(new GenerateReportCardsRequest
            {
                TermId = genTermId.Value,
                SchoolClassId = genClassId.Value,
                NextTermBegins = genNextTerm.HasValue ?
DateOnly.FromDateTime(genNextTerm.Value) : null,
            });

            if (op.Succeeded && op.Data is not null)
            {
                Notification.Notify(NotificationSeverity.Success, "Generated",
                    $"{op.Data.CardsGenerated} new, {op.Data.CardsUpdated} refreshed.");
                showGenerate = false;
                termId = genTermId;
                classId = genClassId;
                await LoadAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
op.Errors));
            }
        }
        finally
        {
            generating = false;
        }
    }
```

```
    private async Task PublishAsync(ReportCardDto c)
    {
        var result = await ReportCardService.PublishAsync(c.Id);
        ShowResult(result, $"Report card for {c.StudentName} published.");
        await LoadAsync();
    }

    private async Task UnpublishAsync(ReportCardDto c)
    {
        var result = await ReportCardService.UnpublishAsync(c.Id);
        ShowResult(result, $"Report card for {c.StudentName} unpublished.");
        await LoadAsync();
    }

    private async Task DeleteAsync(ReportCardDto c)
    {
        var confirm = await Dialog.Confirm(
            $"Delete the report card for {c.StudentName}?",
            "Delete report card",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await ReportCardService.SoftDeleteAsync(c.Id);
        ShowResult(result, "Report card deleted.");
        await LoadAsync();
    }

    private void ShowResult(OperationResult result, string success)
    {
        if (result.Succeeded)
            Notification.Notify(NotificationSeverity.Success, "Done", success);
        else
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
    }
}
```

9.6 ReportCardDetail.razor — the long form

The most substantial page in the sprint. Header summary cards show subjects taken, average, position, days present. Below that, a Radzen tab strip splits the page into Subjects (read-only result table), Affective traits, Psychomotor skills, and Comments. The two ratings tabs render an HTML table per category; choosing a rating from the dropdown auto-saves via UpsertAffectiveRatingAsync / UpsertPsychomotorRatingAsync. The Comments tab is a small form with class-teacher comment, head-teacher comment, and the next-term date.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Results/ReportCardDetail.razor

```
@page "/reports/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IReportCardService ReportCardService
```

```
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Report card · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading report card...</p>
}
else if (detail is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Report card not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div>
            <h1>@detail.Card.StudentName</h1>
            <p>
                @detail.Card.SchoolClassName · @detail.Card.TermName · admission
#@detail.Card.StudentAdmissionNumber
            </p>
        </div>
        <div class="nps-page-actions">
            @if (detail.Card.IsPublished)
            {
                <RadzenBadge Text="Published" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                <RadzenButton Text="Unpublish" Icon="lock_open" Variant="Variant.Flat"
                              ButtonStyle="ButtonStyle.Warning" Click="@UnpublishAsync" />
            }
            else
            {
                <RadzenButton Text="Publish" Icon="lock" Variant="Variant.Flat"
                              ButtonStyle="ButtonStyle.Success" Click="@PublishAsync" />
            }
            <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                          ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/reports"))" />
        </div>
    </div>

    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <div class="nps-stat-grid">
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="auto_stories" /></div>
                <div>
                    <div class="nps-stat-label">Subjects offered</div>
                    <div class="nps-stat-value">@detail.Card.SubjectsTaken</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="percent" /></div>
```

```
                <div>
                    <div class="nps-stat-label">Average</div>
                    <div class="nps-stat-
value">@detail.Card.AveragePercentage.ToString("F2")%</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="emoji_events" /></div>
                <div>
                    <div class="nps-stat-label">Position in class</div>
                    <div class="nps-stat-value">
                        @(detail.Card.Position.HasValue ? $"{detail.Card.Position} of
{detail.Card.StudentsInClass}" : "—")
                    </div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="event_available" /></div>
                <div>
                    <div class="nps-stat-label">Days present</div>
                    <div class="nps-stat-value">@detail.Card.DaysPresent /
@detail.Card.TotalSchoolDays</div>
                </div>
            </div>
        </div>
    </RadzenCard>

    <RadzenTabs @bind-SelectedIndex="selectedTab">
        <Tabs>
            <RadzenTabsItem Text="@($"Subjects ({detail.Results.Count})")">
                <RadzenCard class="nps-card">
                    @if (detail.Results.Count == 0)
                    {
                        <p style="color: var(--nps-ink-500);">
                            No subject results yet. Compute results from the
<strong>Results</strong> page first,
                            then regenerate this card.
                        </p>
                    }
                    else
                    {
                        <RadzenDataGrid TItem="SubjectResultDto" Data="@detail.Results"
AllowPaging="false">
                            <Columns>
                                <RadzenDataGridColumn TItem="SubjectResultDto"
Title="Subject" Property="SubjectName">
                                    <Template Context="r">
                                        <strong>@r.SubjectName</strong>
                                        <br />
                                        <small style="color: var(--nps-
ink-500);">@r.SubjectCode</small>
                                    </Template>
                                </RadzenDataGridColumn>
                                <RadzenDataGridColumn TItem="SubjectResultDto" Title="%"
Width="100px">
```

```
                                    <Template Context="r">
                                        <strong>@r.Percentage.ToString("F2")%</strong>
                                    </Template>
                                </RadzenDataGridColumn>
                                <RadzenDataGridColumn TItem="SubjectResultDto"
Title="Grade" Width="120px">
                                    <Template Context="r">
                                        @if (!string.IsNullOrEmpty(r.GradeBandName))
                                        {
                                            <RadzenBadge Text="@r.GradeBandName"

BadgeStyle="@GradeStyle(r.GradeBandName)"
                                                         Variant="Variant.Flat" />
                                        }
                                    </Template>
                                </RadzenDataGridColumn>
                                <RadzenDataGridColumn TItem="SubjectResultDto"
Title="Position" Width="120px">
                                    <Template Context="r">
                                        @(r.Position.HasValue ? $"{r.Position} /
{r.StudentsInClass}" : "—")
                                    </Template>
                                </RadzenDataGridColumn>
                                <RadzenDataGridColumn TItem="SubjectResultDto"
Title="Remark">
                                    <Template Context="r">
                                        @r.GradeBandRemark
                                    </Template>
                                </RadzenDataGridColumn>
                            </Columns>
                        </RadzenDataGrid>
                    }
                </RadzenCard>
            </RadzenTabsItem>

            <RadzenTabsItem Text="Affective traits">
                <RadzenCard class="nps-card">
                    <p style="color: var(--nps-ink-500); margin: 0 0 1rem;">
                        Pick a rating per trait. Saved automatically as you change each
row.
                    </p>
                    <table class="nps-trait-grid">
                        <thead>
                            <tr>
                                <th>Trait</th>
                                <th style="width: 240px;">Rating</th>
                            </tr>
                        </thead>
                        <tbody>
                            @foreach (var trait in affectiveTraits)
                            {
                                var existing = detail.AffectiveRatings.FirstOrDefault(r =>
r.AffectiveTraitId == trait.Id);
                                <tr>
                                    <td><strong>@trait.Name</strong></td>
```

```
                                    <td>
                                        <RadzenDropDown TValue="Guid?" Data="@traitRatings"
                                                        TextProperty="Name"
ValueProperty="Id"
                                                        Value="@(existing?.TraitRatingId)"
                                                        Disabled="@detail.Card.IsPublished"
                                                        Change="@((object v) =>
SaveAffectiveAsync(trait.Id, v as Guid?))"
                                                        Style="width: 100%;"
Placeholder="—" />
                                    </td>
                                </tr>
                            }
                        </tbody>
                    </table>
                </RadzenCard>
            </RadzenTabsItem>

            <RadzenTabsItem Text="Psychomotor skills">
                <RadzenCard class="nps-card">
                    <table class="nps-trait-grid">
                        <thead>
                            <tr>
                                <th>Skill</th>
                                <th style="width: 240px;">Rating</th>
                            </tr>
                        </thead>
                        <tbody>
                            @foreach (var skill in psychomotorSkills)
                            {
                                var existing = detail.PsychomotorRatings.FirstOrDefault(r
=> r.PsychomotorSkillId == skill.Id);
                                <tr>
                                    <td><strong>@skill.Name</strong></td>
                                    <td>
                                        <RadzenDropDown TValue="Guid?" Data="@traitRatings"
                                                        TextProperty="Name"
ValueProperty="Id"
                                                        Value="@(existing?.TraitRatingId)"
                                                        Disabled="@detail.Card.IsPublished"
                                                        Change="@((object v) =>
SavePsychomotorAsync(skill.Id, v as Guid?))"
                                                        Style="width: 100%;"
Placeholder="—" />
                                    </td>
                                </tr>
                            }
                        </tbody>
                    </table>
                </RadzenCard>
            </RadzenTabsItem>

            <RadzenTabsItem Text="Comments">
                <RadzenCard class="nps-card">
                    <div class="nps-form-grid">
```

```
                        <div class="nps-field nps-span-2">
                            <label>Class teacher's comment</label>
                            <RadzenTextArea @bind-Value="commentsForm.ClassTeacherComment"
Rows="3"
                                            Disabled="@detail.Card.IsPublished" />
                        </div>
                        <div class="nps-field nps-span-2">
                            <label>Head teacher's comment</label>
                            <RadzenTextArea @bind-Value="commentsForm.HeadTeacherComment"
Rows="3"
                                            Disabled="@detail.Card.IsPublished" />
                        </div>
                        <div class="nps-field">
                            <label>Next term begins</label>
                            <RadzenDatePicker TValue="DateTime?" @bind-
Value="commentsForm.NextTermBegins"
                                              Disabled="@detail.Card.IsPublished"
                                              DateFormat="d MMM yyyy" ShowTime="false" />
                        </div>
                    </div>
                    <div class="nps-form-actions">
                        <RadzenButton Text="@(savingComments ? "Saving..." : "Save
comments")"
                                      Icon="save" ButtonStyle="ButtonStyle.Primary"
                                      Disabled="@(savingComments ||
detail.Card.IsPublished)"
                                      Click="@SaveCommentsAsync" />
                    </div>
                </RadzenCard>
            </RadzenTabsItem>
        </Tabs>
    </RadzenTabs>
}

@code {
    [Parameter] public Guid Id { get; set; }

    private bool loading = true;
    private bool savingComments;
    private int selectedTab;

    private ReportCardDetailDto? detail;
    private CommentsFormModel commentsForm = new();

    private IReadOnlyList<LookupDto> affectiveTraits = [];
    private IReadOnlyList<LookupDto> psychomotorSkills = [];
    private IReadOnlyList<LookupDto> traitRatings = [];

    private class CommentsFormModel
    {
        public string? ClassTeacherComment { get; set; }
        public string? HeadTeacherComment { get; set; }
        public DateTime? NextTermBegins { get; set; }
    }
```

```
    protected override async Task OnInitializedAsync()
    {
        affectiveTraits = await LookupService.GetAffectiveTraitsAsync();
        psychomotorSkills = await LookupService.GetPsychomotorSkillsAsync();
        traitRatings = await LookupService.GetTraitRatingsAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            detail = await ReportCardService.GetByIdAsync(Id);
            if (detail is not null)
            {
                commentsForm = new CommentsFormModel
                {
                    ClassTeacherComment = detail.Card.ClassTeacherComment,
                    HeadTeacherComment = detail.Card.HeadTeacherComment,
                    NextTermBegins =
detail.Card.NextTermBegins?.ToDateTime(TimeOnly.MinValue),
                };
            }
        }
        finally
        {
            loading = false;
        }
    }

    private async Task SaveCommentsAsync()
    {
        if (detail is null) return;
        savingComments = true;
        try
        {
            var result = await ReportCardService.UpdateCommentsAsync(new
UpdateReportCardCommentsRequest
            {
                Id = Id,
                ClassTeacherComment = commentsForm.ClassTeacherComment,
                HeadTeacherComment = commentsForm.HeadTeacherComment,
                NextTermBegins = commentsForm.NextTermBegins.HasValue
                    ? DateOnly.FromDateTime(commentsForm.NextTermBegins.Value) : null,
            });
            ShowResult(result, "Comments saved.");
            if (result.Succeeded) await LoadAsync();
        }
        finally
        {
            savingComments = false;
        }
    }
```

```
    private async Task SaveAffectiveAsync(Guid traitId, Guid? ratingId)
    {
        if (detail is null || ratingId is null) return;

        var result = await ReportCardService.UpsertAffectiveRatingAsync(new
UpsertAffectiveRatingRequest
        {
            ReportCardId = Id,
            AffectiveTraitId = traitId,
            TraitRatingId = ratingId.Value,
        });
        ShowResult(result, "Trait rating saved.");
        if (result.Succeeded) await LoadAsync();
    }

    private async Task SavePsychomotorAsync(Guid skillId, Guid? ratingId)
    {
        if (detail is null || ratingId is null) return;

        var result = await ReportCardService.UpsertPsychomotorRatingAsync(new
UpsertPsychomotorRatingRequest
        {
            ReportCardId = Id,
            PsychomotorSkillId = skillId,
            TraitRatingId = ratingId.Value,
        });
        ShowResult(result, "Skill rating saved.");
        if (result.Succeeded) await LoadAsync();
    }

    private async Task PublishAsync()
    {
        var confirm = await Dialog.Confirm(
            "Publish this report card? Once published the comments and ratings can no
longer be edited.",
            "Publish report card",
            new ConfirmOptions { OkButtonText = "Publish", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await ReportCardService.PublishAsync(Id);
        ShowResult(result, "Report card published.");
        if (result.Succeeded) await LoadAsync();
    }

    private async Task UnpublishAsync()
    {
        var result = await ReportCardService.UnpublishAsync(Id);
        ShowResult(result, "Report card unpublished.");
        if (result.Succeeded) await LoadAsync();
    }

    private void ShowResult(OperationResult result, string success)
    {
```

```
        if (result.Succeeded)
            Notification.Notify(NotificationSeverity.Success, "Done", success);
        else
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
    }

    private static BadgeStyle GradeStyle(string? grade) => grade switch
    {
        "A" => BadgeStyle.Success,
        "B" => BadgeStyle.Info,
        "C" => BadgeStyle.Primary,
        "D" => BadgeStyle.Warning,
        "E" => BadgeStyle.Warning,
        "F" => BadgeStyle.Danger,
        _ => BadgeStyle.Secondary,
    };
}
```

10. Navigation, imports, and authorization

10.1 NavMenu — a new Results & Reports panel

NavMenu.razor picks up a fifth role-gated panel between Attendance and the Finance/Inventory placeholders. Three entries live inside it: Assessments, Subject results, Report cards. The panel is wrapped in an AuthorizeView that requires SuperAdmin, HeadTeacher, or Teacher.

Excerpt — NavMenu.razor

```
            <RadzenPanelMenuItem Text="Results & Reports" Icon="assignment"
Expanded="true">
                <RadzenPanelMenuItem Text="Assessments" Icon="grading"
Path="/assessments" />
                <RadzenPanelMenuItem Text="Subject results" Icon="leaderboard"
Path="/results" />
                <RadzenPanelMenuItem Text="Report cards" Icon="description" Path="/reports"
/>
            </RadzenPanelMenuItem>
        </Authorized>
    </AuthorizeView>
```

10.2 _Imports.razor

_Imports.razor picks up two new @using lines so the results DTOs and service interfaces are visible to every Razor file in the Web project.

Excerpt — _Imports.razor

```
@using NaijaPrimeSchool.Application.Results
@using NaijaPrimeSchool.Application.Results.Dtos
```

10.3 Authorization at the page level

Every sprint-5 page declares an [Authorize] attribute. Assessment pages require SuperAdmin + HeadTeacher + Teacher; result and report-card pages require SuperAdmin + HeadTeacher. There is no policy added — the existing role attribute is enough. Future sprints (parent portal, student portal) will introduce a Pastoral or Family policy that exposes published report cards to parents.

11. Lifecycle of a results pipeline run

Walking a single (term, class) from blank gradebook to published report cards is the clearest way to see how all the layers cooperate.

11.1 Setting up the gradebook

- HeadTeacher opens /assessments. Filter to the current term and the target class.
- Click New assessment. Pick the subject, pick CA1 from the type dropdown, set max score 20 and weight 1, click Save.
- Repeat for CA2 (max 20, weight 1) and Examination (max 60, weight 2). The grid now lists three draft assessments for the subject.

11.2 Entering scores

- Open the score sheet for CA1. Every actively-enrolled pupil is pre-listed. Key in scores; mark a pupil absent if they did not sit the test.
- Click Save scores. BulkSetScoresAsync writes the rows in one round-trip; ApplicationDbContext.SaveChanges stamps CreatedOn/By on each new row.
- Repeat for CA2 and Examination.
- Optionally Publish each assessment when scores are final. Published assessments are read-only.

11.3 Computing subject results

- Open /results, pick the same term and class.
- Click Recompute. ResultService.ComputeAsync produces one SubjectResult per (pupil, subject) and one warning per skipped finalised row, if any.
- The grid now shows percentages, grade badges, and positions out of the cohort size.
- When the head teacher is satisfied, click Compute & finalise. The same compute runs and rows are stamped FinalisedOn = now. Further recomputes will not touch them unless reopened.

11.4 Composing report cards

- Open /reports, click Generate / refresh, pick the same (term, class), set the next-term-begins date, click Generate.
- ReportCardService.GenerateAsync rolls every SubjectResult for the (term, class), joins to the sprint-4 attendance counts, and ranks pupils by average percentage.
- Click any row. The detail page tabs split profile / subjects / ratings / comments.
- Pick affective and psychomotor ratings; type the class teacher's comment; type the head teacher's comment.
- Publish. Card is now read-only and ready for the parent portal sprint to surface.

11.5 Correcting an error after publishing

- A teacher notices a score was wrong. Open the report card, click Unpublish.
- Open /results, find the affected SubjectResult, click Reopen.
- Open the underlying assessment, unpublish it, fix the score, save.
- Re-publish the assessment, run Recompute on the results page. The single row updates.

- Run Generate / refresh on /reports — the existing card row refreshes (it was unpublished, so the generator does write to it).
- Re-publish the card. Audit columns (ModifiedOn/By) reflect the chain of edits.

12. Smoke-test walkthrough

Once the build is green and the migration has applied, this is the end-to-end smoke test for a fresh checkout.

12.1 Build, migrate, run

```
dotnet restore
dotnet build NaijaPrimeSchool.slnx
dotnet run --project src/NaijaPrimeSchool.Web
```

First run applies migrations and seeds the five new lookup tables. Sign in as superadmin@naijaprimeschool.ng / Admin@12345.

12.2 Verify navigation

- The Results & Reports panel appears in the sidebar, between Attendance and the Finance placeholder.
- It contains three items: Assessments, Subject results, Report cards.
- Each one renders without error.

12.3 Verify the seeded lookups

Connect to SQL Server and run:

```
SELECT Name, Code, DefaultMaxScore, IsExam FROM AssessmentTypes ORDER BY DisplayOrder;
SELECT Name, LowerBound, UpperBound, Description FROM GradeBands ORDER BY DisplayOrder;
SELECT Name FROM AffectiveTraits ORDER BY DisplayOrder;
SELECT Name FROM PsychomotorSkills ORDER BY DisplayOrder;
SELECT Name, Value FROM TraitRatings ORDER BY DisplayOrder;
```

12.4 Create assessments, score them, compute results

16. Pre-requisites — at least one Term, one SchoolClass with actively enrolled pupils, and at least one Subject. (Sprints 2–3 cover the setup.)
17. Create three assessments (CA1, CA2, Exam) for one subject in the class.
18. Open each score sheet, key scores for every pupil, save.
19. Open /results, pick the term and class, click Recompute. Verify the percentages and positions look right.
20. Click Compute & finalise on the same row. Verify the rows flip to the Finalised badge.

12.5 Generate and publish report cards

21. Open /reports, click Generate / refresh, pick the same (term, class), generate.
22. Open one card. Verify the subject totals match what /results showed. Verify the attendance counts match a quick spot-check against the sprint-4 register.
23. Pick affective and psychomotor ratings; type teacher and head-teacher comments.
24. Click Publish. Verify the action buttons disable and the status badge flips to Published.

12.6 Verify error paths

25. Try to delete a published assessment. The OperationResult comes back with 'Unpublish the assessment before deleting.'
26. Try to delete a finalised SubjectResult. The OperationResult comes back with 'Cannot delete a finalised result. Reopen it first.'
27. Try to delete a published ReportCard. The OperationResult comes back with 'Unpublish the card before deleting.'
28. Try to enter a score larger than the assessment's MaxScore. The score endpoint rejects it with the bounded-range message.

13. Troubleshooting and gotchas

13.1 'No assessments exist for this term/class'

Recompute fails with this message when the (term, class) has zero TermAssessment rows. Create assessments under /assessments first.

13.2 'No subject results exist for this term/class'

Generate report cards fails when results have not been computed yet. Open /results, pick the same (term, class), click Recompute, then come back.

13.3 The score sheet shows pupils I did not enrol

GetScoreSheetAsync pre-lists every pupil with an active enrolment in the assessment's class. If a pupil who left still appears, their enrolment row is still open. Withdraw the enrolment under Family → Students → pupil → Enrolment history, then reload the score sheet.

13.4 Position is the same for two pupils

Dense ranking gives ties the same position number. If two pupils have 78.50% they are both '5 of 38'. The next pupil is '6 of 38' (not 7). This is intentional and matches the way most schools rank. If you need 1224-style ranking, the ResultService.ComputeAsync method is the place to change it.

13.5 'Score must be between 0 and N'

Every score is bounded by the assessment's MaxScore. If MaxScore is 20, valid scores are 0..20 inclusive. Rounding (20.0001 = 20) is not done; either edit the MaxScore or lower the entered score.

13.6 'Already finalised — skipped recompute'

ResultService.ComputeAsync warns rather than fails when it encounters a finalised SubjectResult during a non-finalising recompute. The warning identifies the (student, subject) so the user can reopen the right row before recomputing.

13.7 The IdentityRole migration warning

The EF Core warning about ApplicationRole's query filter and IdentityUserRole's required end is harmless at runtime and predates sprint 5. The build is still green.

14. Forward-compatibility, today

Sprint 5 has left a few breadcrumbs that make later sprints easier.

- ReportCard.IsPublished is the load-bearing flag the parent and student portals will check. Until that sprint, only internal staff can see cards.
- SubjectResult.IsFinalised distinguishes draft from authoritative rows, so a future analytics dashboard can choose to show only finalised data.
- GradeBand.LowerBound/UpperBound use decimal(5,2). Adding a new band (e.g. A+ for 90+) is a one-row insert plus a re-seed guard, no schema change.
- AssessmentType.IsExam is reserved for end-of-session promotion logic — 'pupil passes the term if their exam mean is at least 50%' will read this column.
- TermAssessment.Weight defaults to 1m, so existing schemes remain valid as the school evolves its grading policy.
- ReportCard.NextTermBegins is a date the parent portal will show next to the published card so families know when the child returns.

14.1 What might need a small refactor later

- ResultService.ComputeAsync issues four queries (assessments, scores, enrolments, grade bands) before its in-memory loop. For a school of 5,000 pupils this stays fine; if a school scales to 20,000+ we may want to push the loop into a SQL stored procedure or a CTE-driven single query.
- BulkSetScoresAsync currently writes one row per Add. EF Core 10's bulk operations would be faster but the API surface changes. Worth doing the day someone reports a slow save.
- Report cards are persisted with the 'next term begins' date as a flat column. If that policy ever varies per pupil (unlikely) we'd promote it to its own table; for now the denormalisation matches every Nigerian school I have built for.
- The HTML score-grid is plain HTML rather than RadzenDataGrid. If a future feature wants column sorting on this grid, we will rewrite it as a Radzen grid with a custom edit template; the back-end DTOs are already in the right shape.

15. Licence

Naija Prime School is released under the MIT Licence. The full text lives in the LICENSE file at the repository root and is embedded below for completeness so the guide stays a single self-contained document.

In short: anyone who obtains a copy of the software may use, copy, modify, merge, publish, distribute, sublicense, or sell it, on the single condition that the copyright notice and the permission notice are preserved. The software is provided "as is", without warranty of any kind. The licence is a deliberate signal that this codebase is meant to be reused — by another Nigerian primary school, by a teacher who wants a weekend project, by a student who wants a portfolio piece — and it lifts the legal friction of doing so.

15.1 Why MIT

- Compatibility. MIT is permissive and compatible with virtually every other licence used by the .NET ecosystem libraries this project depends on (Radzen Blazor, EF Core, ASP.NET Core Identity, etc.).
- Familiarity. Most Nigerian developers and many international OSS contributors recognise MIT immediately; a permissive licence reduces hesitation to fork or contribute.
- Brevity. The text fits on a single page, so future maintainers do not have to read a long policy document to understand what they can and cannot do.
- No copyleft. Schools that decide to extend the platform with in-house features are not forced to publish those changes. This matches the realities of school IT teams.

15.2 Full text of the LICENSE file

LICENSE

```
MIT License

Copyright (c) 2026 Benjamin Fadina

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

The README at the repository root carries a short License section that points back at this file; the solution file (NaijaPrimeSchool.slnx) lists LICENSE alongside README.md and the per-sprint guides under Solution Items so it is visible in Visual Studio's Solution Explorer.

16. Appendix — files added or changed in sprint 5

Domain layer (new)	—
src/NaijaPrimeSchool.Domain/Results/AssessmentType.cs	Lookup.
src/NaijaPrimeSchool.Domain/Results/GradeBand.cs	Lookup with bounds + remarks.
src/NaijaPrimeSchool.Domain/Results/AffectiveTrait.cs	Lookup.
src/NaijaPrimeSchool.Domain/Results/PsychomotorSkill.cs	Lookup.
src/NaijaPrimeSchool.Domain/Results/TraitRating.cs	Lookup, 1–5.
src/NaijaPrimeSchool.Domain/Results/TermAssessment.cs	Gradebook entry.
src/NaijaPrimeSchool.Domain/Results/AssessmentScore.cs	Per-pupil score.
src/NaijaPrimeSchool.Domain/Results/SubjectResult.cs	Per (pupil, term, subject) total.
src/NaijaPrimeSchool.Domain/Results/ReportCard.cs	Per (pupil, term) summary.
src/NaijaPrimeSchool.Domain/Results/AffectiveRating.cs	Card x trait rating.
src/NaijaPrimeSchool.Domain/Results/PsychomotorRating.cs	Card x skill rating.
Domain layer (modified)	—
src/NaijaPrimeSchool.Domain/Academics/Subject.cs	Added TermAssessments + SubjectResults.
src/NaijaPrimeSchool.Domain/Academics/Term.cs	Added TermAssessments + SubjectResults + ReportCards.
src/NaijaPrimeSchool.Domain/Academics/SchoolClass.cs	Added TermAssessments + SubjectResults + ReportCards.
src/NaijaPrimeSchool.Domain/Family/Student.cs	Added AssessmentScores + SubjectResults + ReportCards.
Application layer (new)	—
src/NaijaPrimeSchool.Application/Results/Dtos/TermAssessmentDtos.cs	TermAssessment DTOs.
src/NaijaPrimeSchool.Application/Results/Dtos/AssessmentScoreDtos.cs	Score-sheet DTOs.
src/NaijaPrimeSchool.Application/Results/Dtos/SubjectResultDtos.cs	Subject result DTOs + compute request/response.
src/NaijaPrimeSchool.Application/Results/Dtos/ReportCardDtos.cs	Report card DTOs + ratings + generate request.
src/NaijaPrimeSchool.Application/Results/IAssessmentService.cs	Assessment + score service contract.
src/NaijaPrimeSchool.Application/Results/IResultService.cs	Result service contract.
src/NaijaPrimeSchool.Application/Results/	Report card service contract.

IReportCardService.cs	
Application layer (modified)	—
src/NaijaPrimeSchool.Application/Users/ ILookupService.cs	Added 5 new lookup methods.
Infrastructure layer (new)	—
src/NaijaPrimeSchool.Infrastructure/Services/ AssessmentService.cs	Assessment CRUD + score-sheet methods.
src/NaijaPrimeSchool.Infrastructure/Services/ ResultService.cs	Compute, finalise, reopen.
src/NaijaPrimeSchool.Infrastructure/Services/ ReportCardService.cs	Generate, comments, ratings, publish.
src/NaijaPrimeSchool.Infrastructure/ Persistence/Migrations/ 20260504172028_AssessmentsAndResults.cs	EF migration adding 11 tables and indexes.
Infrastructure layer (modified)	—
src/NaijaPrimeSchool.Infrastructure/ DependencyInjection.cs	Registered the 3 new services.
src/NaijaPrimeSchool.Infrastructure/ Persistence/ApplicationDbContext.cs	Added 11 DbSets, ConfigureResults.
src/NaijaPrimeSchool.Infrastructure/ Persistence/DatabaseInitializer.cs	Seeded the 5 new lookup tables.
src/NaijaPrimeSchool.Infrastructure/Services/ LookupService.cs	Added 5 new lookup methods.
Web layer (new)	—
src/NaijaPrimeSchool.Web/Components/ Pages/Results/Assessments.razor	Gradebook list + inline form.
src/NaijaPrimeSchool.Web/Components/ Pages/Results/AssessmentScores.razor	Score sheet.
src/NaijaPrimeSchool.Web/Components/ Pages/Results/Results.razor	Compute, view, finalise.
src/NaijaPrimeSchool.Web/Components/ Pages/Results/ReportCards.razor	Generate + list.
src/NaijaPrimeSchool.Web/Components/ Pages/Results/ReportCardDetail.razor	Tabs: subjects / traits / skills / comments.
Web layer (modified)	—
src/NaijaPrimeSchool.Web/Components/ _Imports.razor	Added Results + Results.Dtos usings.
src/NaijaPrimeSchool.Web/Components/ Layout/NavMenu.razor	Added the Results & Reports panel.
src/NaijaPrimeSchool.Web/wwwroot/app.css	Added .nps-score-grid and .nps-trait-grid styles.
Tooling (new)	—
tools/generate_sprint5_guide.py	This document's generator.
Project root (modified)	—
LICENSE	MIT licence — added to formalise the open-source release.
README.md	License section now points at the MIT LICENSE file.
NaijaPrimeSchool.slnx	LICENSE registered under Solution Items so it is

	visible in Solution Explorer.

— End of the Sprint 5 implementation guide. The next sprint lands fees and bursar workflows on top of the (Pupil x Term) primitives established here.